

Rapport de Projet de Fin d'Années

Présenté en vue de l'obtention du
Diplôme National d'Ingénieur en Systèmes embarqués et objets connectés

Par

Khelifi Nourhen

Solution embarquée IoT pour le monitoring des paramètres critiques aéronautiques et la détection d'anomalies

Encadrant professionnel :

Dr. Jamal ETTOUSY

Réalisé au sein de



Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du
Diplôme National d'Ingénieur en Systèmes embarqués et objets connectés

Par

Khelifi Nourhen

Solution embarquée IoT pour le monitoring des paramètres critiques aéronautiques et la détection d'anomalies

Encadrant professionnel : **Dr. Jamal ETTOUSY**

.....

Réalisé au sein de SIGA



J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant professionnel, **Dr. Jamal ETTOUSY**

Signature et cachet

Dédicaces

À ceux qui ont fait de moi ce que je suis aujourd'hui...

Je dédie ce modeste travail à :

À mes chers parents,

pour leur amour inconditionnel, leurs sacrifices et leur soutien indéfectible tout au long de ce parcours. Aucun mot ne saurait exprimer ma gratitude et mon profond respect. Que Dieu vous bénisse et vous garde.

À ma famille et mes proches,

pour leur encouragement constant et leur présence à mes côtés dans les moments les plus importants.

À tous mes amis et collègues,

pour les moments partagés, les échanges enrichissants et le soutien mutuel qui ont rendu cette aventure plus agréable.

À tous ceux qui, de près ou de loin, ont contribué à la réalisation de ce travail,
je leur exprime ma profonde gratitude.

Khelifi Nourhen

Remerciements

<La gratitude est la mémoire du cur. >

Je tiens à exprimer mes sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

À Monsieur Jamal ETTOUSY, mon encadrant professionnel,
pour son accompagnement, ses précieux conseils et son soutien indéfectible tout au long de ce projet. Sa rigueur, sa disponibilité et sa bienveillance ont été pour moi une source d'inspiration et de motivation.

À mon encadrant académique,
pour ses orientations avisées, son suivi attentif et la confiance qu'il m'a accordée.

À tous les membres de l'équipe de YaneCode Digital,
pour leur accueil chaleureux, leur esprit de collaboration et les échanges enrichissants partagés durant cette expérience.

À ma famille et mes proches,
pour leur soutien inconditionnel, leurs encouragements et leur patience tout au long de ce parcours.

À mes amis et camarades,
pour les moments de partage, d'entraide et de bonne humeur qui ont rendu cette aventure plus agréable.

À tous ceux qui, par leur soutien ou leurs conseils, ont contribué à
l'aboutissement de ce travail,
qu'ils trouvent ici l'expression de ma profonde gratitude.

Khelifi Nourhen

Table des matières

Dédicaces	i
Remerciements	ii
Introduction générale	1
1 Contexte général	3
1.1 Présentation de l'organisme d'accueil	4
1.2 Contexte général du projet	6
1.2.1 Problématique	6
1.3 Étude et critique de l'existant	7
1.3.1 Analyse des solutions existantes de monitoring	7
1.3.2 Critique de l'existant	8
1.4 Présentation du cahier des charges	9
1.4.1 Description du projet	9
1.4.2 Objectif du projet	9
1.4.3 Résultats attendus	10
1.5 Méthodologie adoptée	10
1.5.1 Présentation de la méthodologie	10
1.5.2 Justification du choix	10
1.6 Application des phases au projet	11
1.7 Planification du projet	13
2 État de l'art et justification des choix technologiques	16
2.1 Systèmes IoT et architectures embarquées pour le monitoring	17
2.1.1 Définition de l'IoT	17
2.1.2 Architecture IoT	17
2.1.3 Systèmes embarqués dans les applications critiques	18
2.1.4 Analyse des architectures embarquées existantes	18
2.2 Cartes embarquées et plateformes de traitement	20

2.2.1	ESP32	20
2.2.2	STM32	20
2.2.3	Raspberry Pi	21
2.2.4	Comparaison des plateformes	21
2.2.5	Justification du choix de la plateforme	21
2.3	Capteurs et paramètres critiques aéronautiques	22
2.3.1	Température	22
2.3.2	Pression	22
2.3.3	Vibration et mouvement	23
2.3.4	Comparaison des capteurs étudiés	23
2.3.5	Justification des paramètres surveillés	23
2.4	Acquisition et prétraitement des données	24
2.4.1	Acquisition des données capteurs	24
2.4.2	Bruit et perturbations des mesures	24
2.4.3	Techniques de filtrage	24
2.4.4	Justification du choix du filtrage	26
2.5	Détection d'anomalies et IA en Edge	26
2.5.1	Méthodes de Machine Learning étudiées	26
2.5.2	Comparaison des méthodes d'IA et Datasets	27
2.5.3	Justification	27
2.6	Protocoles de communication IoT	27
2.6.1	Comparaison des protocoles (WiFi, Bluetooth, UART, MQTT	27
2.6.2	Justification du choix MQTT	28
2.6.3	Justification du choix MQTT	28
3	Analyse des besoins et conception de la solution	30
3.1	Introduction	31
3.2	Identification des besoins	31
3.2.1	Besoins fonctionnels	31
3.2.2	Besoins non fonctionnels	32
3.3	Identification des acteurs du système	32
3.4	Architecture détaillée du système	34

3.4.1	Architecture matérielle	34
3.4.2	Architecture logicielle	36
3.4.3	Flux de données	37
3.5	Conception UML	39
3.5.1	Diagramme de cas d'utilisation	39
3.5.2	Diagramme de classes	40
3.5.3	Diagrammes de séquence	40
3.6	Conclusion	44
4	Réalisation et validation	45
4.1	Introduction	46
4.2	Environnement matériel et logiciel	46
4.2.1	Environnement matériel	46
4.2.2	Environnement logiciel	48
4.2.3	Langages de programmation	55
4.3	Implémentation logicielle	56
4.3.1	Acquisition et traitement des données	56
4.3.2	Gestion des communications MQTT	60
4.3.3	Gestion des erreurs et logging	61
4.4	Intelligence artificielle pour la détection des anomalies	62
4.4.1	Objectif de la détection par IA	62
4.4.2	Données utilisées pour le modèle	62
4.4.3	Séparation temporelle des données	65
4.4.4	Analyse et préparation des données	65
4.4.5	Séparation temporelle des données	66
4.4.6	Prétraitement des données	67
4.4.7	Ingénierie des caractéristiques	67
4.4.8	Réduction dimensionnelle par PCA	67
4.4.9	Modèle Isolation Forest	68
4.4.10	Modèle One-Class SVM	68
4.4.11	Autoencodeur LSTM	69
4.4.12	Construction des séquences temporelles	70

4.4.13	Validation des modèles	70
4.4.14	Résultats obtenus	70
4.4.15	Comparaison des modèles	71
4.4.16	Matrices de confusion et courbes d'évaluation	72
4.4.17	Export du modèle pour le déploiement	74
4.4.18	Intégration de l'IA dans l'architecture IoT	75
4.4.19	Stratégie finale de détection	75
4.5	Conclusion	75
Conclusion générale		77
Bibliographie		79

Table des figures

1.1	:Logo YaneCode	5
1.2	Logo YaneCode Robotique & Academy	5
1.3	Acteur systeme	13
1.4	Interfaces de suivi de lavancement du projet	14
1.5	Interfaces de planification du projet	15
3.1	Acteur systeme	34
3.2	Acteur systeme	35
3.3	edge flux de donnes	37
3.4	edge flux de donnes	38
3.5	edge flux de donnes	39
3.6	edge flux de donnes	40
3.7	Diagramme d'Acquisition et transmission en temps réel	41
3.8	Diagramme Séquence de détection d'une anomalie	42
3.9	Diagramme Séquence de Consultation de l'historique	43
4.1	edge flux couche middlwre	46
4.2	Logo Wokwi	47
4.3	Cycle de traitement des données sur IESP32	47
4.4	Logo slack	48
4.5	Logo HiveMQ	49
4.6	Processus de validation et de traitement des trames MQTT	49
4.7	Chaîne Middleware de traitement des données IoT	50
4.8	Logo InfluxDB	50
4.9	Processus de stockage et dexploitation des données IoT	52
4.10	Logo ngrok	53
4.11	Flux de données entre InfluxDB et le tableau de bord Grafana	54
4.12	interface de tableau de bord Grafana	55
4.13	Logs de connexion et publication MQTT sur le moniteur série	61
4.14	Statistiques exploratoires	65

4.15	Statistiques exploratoires	66
4.16	Réduction dimensionnelle par analyse en composantes principales	68
4.17	Comparaison multicritère des modèles de détection d'anomalies	72
4.18	Courbes ROC des quatre modèles	73
4.19	Courbes Precision-Recall des quatre modèles	73
4.20	Matrices de confusion des modèles	74
4.21	Comparaison des courbes ROC	74
4.22	Comparaison des courbes Precision-Recall	74

Liste des tableaux

2.1	Comparaison des plateformes embarquées	21
2.2	Comparaison des capteurs environnementaux	23
2.3	Comparaison des protocoles de communication IoT	28
2.4	Sélection de l'écosystème logiciel	28
3.1	Identification des acteurs du système	33
3.2	Architecture matérielle et périphériques associés à l'ESP32	35
4.1	Bibliothèques logicielles utilise dans la simulation	48
4.2	Schéma de la base de données InfluxDB 3 Core	51
4.3	Taux d'anomalie par scénario de défaillance	64
4.4	Découpage temporel des données	65
4.5	Découpage temporel des données	66
4.6	Résultats des modèles sur l'ensemble de test	71
4.7	Matrice de confusion – Isolation Forest	74
4.8	Matrice de confusion – One-Class SVM	74

Liste des abréviations

—	ADC	=	A nalog- t o- D igital C onverter Convertisseur Analogique-Numérique
—	API	=	A pplication P rogramming I nterface (Interface de Programmation d'Applications)
—	CPU	=	C entral P rocessing U nit Unité Centrale de Traitement
—	CoAP	=	C onstrained A pplication P rotocol (Protocole pour les objets contraints)
—	GPIO	=	G eneral P urpose I nput/ O utput (Entrée/Sortie à Usage Général)
—	GUI	=	G raphical U ser I nterface (Interface Utilisateur Graphique)
—	Grafana	=	Grafana (Plateforme open-source de visualisation et d'analyse de données)
—	HTTP	=	H yper T ext T ransfer P rotocol (Protocole de transfert hypertexte)
—	HiveMQ	=	HiveMQ (Broker MQTT professionnel et public)
—	I2C	=	I nter- I ntegrated C ircuit Bus de communication série
—	IA	=	I ntelligence A rtificielle
—	IIoT	=	I ndustrial I nternet o f T hings Internet des Objets Industriel
—	InfluxDB	=	I nflux D B Base de données open-source optimisée pour les séries temporelles
—	JSON	=	J ava S cript O bject N otation (Format léger d'échange de données)
—	LSTM	=	L ong S hort- T erm M emory (Mémoire Longue Court-Terme - type de réseau de neurones récurrent)
—	ML	=	M achine L earning (Apprentissage Automatique)
—	MQTT	=	M essage Q ueuing T elemetry T ransport (Protocole de communication léger pour l'IoT)
—	NTP	=	N etwork T ime P rotocol (Protocole de synchronisation horaire réseau)
—	Node-RED	=	N ode- R ED (Outil de programmation visuelle open-source pour l'orchestration de flux IoT)
—	PCA	=	P rincipal C omponent A nalysis (Analyse en Composantes Principales)
—	RAM	=	R andom A ccess M emory (Mémoire Vive)
—	RTOS	=	R ead- T ime O perating S ystem (Système d'Exploitation Temps Réel)
—	SPI	=	S erial P eripheral I nterface (Interface périphérique série)
—	SoC	=	S ystem o n a C hip (Système sur une Puce)
—	TLS	=	T ransport L ayer S ecurity (Sécurité de la Couche Transport - protocole de chiffrement)
—	UART	=	U niversal A synchronous R eceiver- T ransmitter (Émetteur-Récepteur Asynchrone Universel)
—	UML	=	U nified M odeling L anguage (Langage de Modélisation Unifié)

- **Wi-Fi** = **Wireless Fidelity** (Fidélité Sans Fil)
- **Wokwi** = **Wokwi** (Simulateur en ligne pour projets de microcontrôleurs)
- **hPa** = **Hectopascal** (Unité de mesure de pression atmosphérique)
- **ngrok** = **ngrok** (Outil de tunneling pour exposer des services locaux)

Introduction générale

La promotion de l'Internet des Objets (IoT) et la miniaturisation des systèmes embarqués ont profondément transformé de nombreux secteurs industriels, et le domaine aéronautique n'y fait pas exception. La possibilité d'équiper les systèmes pour recueillir des données en temps réel et de les analyser localement pave la route vers une nouvelle ère d'applications essentielles, allant de la maintenance prédictive à l'amélioration des performances opérationnelles. Comme l'indiquent les études récentes, l'implémentation de réseaux de capteurs sans fils dans les aéronefs constitue un défi crucial, surtout pour alléger le poids et le filage tout en renforçant la fiabilité des systèmes de monitoring (CSEM et al., projet FLITE-WISE, 2013).

De plus, la normalisation des communications sans fil à l'intérieur de l'avion (WAIC) semble être une approche prometteuse pour relever ces défis (Huang et al., 2025). Dans ce contexte, le monitoring des paramètres de vol et de fonctionnement des équipements est devenu un enjeu majeur de sécurité, de fiabilité et d'efficacité.

L'importance du monitoring des paramètres critiques dans les systèmes aéronautiques réside dans sa capacité à prévenir les défaillances, à anticiper les maintenances et à garantir la sécurité des vols. Historiquement, l'analyse de ces données s'effectuait de manière différée, après l'atterrissage, ou reposait sur des systèmes de surveillance centralisés. Cependant, ces approches présentent des limites en termes de réactivité et de granularité, notamment face à la complexité croissante des aéronefs modernes. Les travaux de recherche récents mettent en évidence le potentiel des systèmes IoT embarqués pour la surveillance de la santé structurelle des aéronefs en temps réel, ouvrant la voie à des approches plus réactives et localisées (Loughborough University, 2025). C'est pourquoi la problématique de la détection précoce des comportements atypiques se pose avec une acuité particulière. Être capable d'identifier une dérive ou une anomalie au moment où elle se produit, sur le système lui-même, représente un gain considérable pour la sécurité et la disponibilité des équipements. La convergence entre l'IoT et l'intelligence artificielle permet aujourd'hui d'envisager une transition de la simple surveillance vers une gestion proactive de la santé des aéronefs (Smith et al., 2024), comme le démontrent des systèmes embarqués intégrant la détection d'anomalies en temps réel et la maintenance prédictive (Chen et al., 2026).

C'est dans ce cadre que s'inscrit la présente mission. Le projet, réalisé au sein de YaneCode Digital, vise à concevoir et développer un prototype de solution IoT embarquée pour le monitoring de paramètres critiques aéronautiques. L'objectif principal est de prouver la viabilité d'un système capable de récupérer des données provenant de capteurs (ou de simulations), de les analyser sur place pour détecter d'éventuelles anomalies par le biais de méthodes simples, et de diffuser les renseignements adaptés à une interface de visualisation. Ce projet vise à satisfaire à un besoin spécifique d'architecture solide, fiable et peu latente, conforme aux exigences des systèmes incorporés, comme indiqué dans le livre de référence de Marilyn Wolf sur l'élaboration des systèmes embarqués (Wolf, 2012).

Pour mener à bien ce projet, une méthodologie structurée a été adoptée. Elle repose sur les principes du Kanban et se décline en plusieurs phases successives : une étude de l'existant et un benchmarking des technologies, la définition précise des besoins et des paramètres à surveiller, la conception de l'architecture matérielle et logicielle, l'implémentation des différents modules, et enfin une phase de tests et de validation rigoureuse. Cette approche itérative et documentée garantit la maîtrise du cycle de vie du projet et la qualité du livrable final.

Le présent rapport détaille l'ensemble des travaux réalisés. Il s'organise comme suit. Le premier chapitre présente le contexte général du projet, incluant l'organisme d'accueil, la problématique, une analyse des solutions existantes et le cahier des charges qui en découle. Le second chapitre expose l'état de l'art et justifie les choix technologiques opérés pour la carte embarquée, les capteurs, les méthodes de filtrage, de détection d'anomalies et les protocoles de communication. Le troisième chapitre est dédié à l'analyse détaillée des besoins et à la conception de la solution, avec une description de l'architecture matérielle, logicielle et des flux de données. Enfin, le quatrième chapitre rend compte de la phase de réalisation, en décrivant l'implémentation de chaque module, les tests effectués et les résultats obtenus, avant de conclure sur les perspectives d'amélioration de ce prototype.

Contexte général

Introduction

Dans ce chapitre, nous présentons le contexte général du projet à travers une description de l'organisme d'accueil, YaneCode Digital, ainsi que de ses filiales YaneCode Robotic et YaneCode Academy. Par la suite, nous détaillons la question principale liée à la supervision des paramètres essentiels en aéronautique, en mettant en lumière les faiblesses majeures des dispositifs en place, comme le manque de centralisation, l'insuffisance de la visualisation et la dépendance au réseau. On procède à une étude approfondie des solutions en place et des infrastructures intégrées pour soutenir l'option d'une démarche décentralisée conforme aux exigences opérationnelles. Nous exposons ensuite les spécifications du projet en énonçant ses buts, ses besoins fonctionnels et techniques, de même que les résultats escomptés. En définitive, nous exposons l'approche Kanban mise en oeuvre, sa mise en pratique spécifique et la planification générale représentée par un graphique de Gantt.

1.1 Présentation de l'organisme d'accueil

Yanecode Digital est une entreprise numérique basée à Kénitra, spécialisée dans l'innovation et l'accompagnement digital. Elle propose une gamme complète de services technologiques, lui assurant une position compétitive sur le marché[1].

Pour répondre aux besoins variés de ses clients, YaneCode Digital a développé une offre de services structurée autour de plusieurs pôles d'expertise :

- **Développement web** : Ce service couvre la conception, la création de contenu, l'hébergement et le suivi de sites web.
- **Développement Desktop** : Il s'agit de la création d'applications installées sur PC, destinées à la digitalisation des services.
- **Développement mobile** : Ce pôle concerne la conception d'applications mobiles pour Android et iOS, permettant un suivi rapproché des activités.
- **E-commerce** : YaneCode Digital assure la création de boutiques en ligne, en intégrant stratégie marketing et hébergement web, comme en témoigne YaneCode Robotic.
- **IoT (Internet of Things)** : L'offre inclut des solutions robotiques et l'organisation de workshops, en lien avec YaneCode Academy, pour l'intégration de l'intelligence artificielle.

- **Cloud** : Ce service met à disposition des infrastructures, plateformes et logiciels accessibles à distance, sans installation supplémentaire.
- **SalesForce** : Il s'agit d'une solution CRM cloud offrant une vue unifiée des clients pour les départements marketing, ventes, service client et e-commerce.
- **Motion graphique** : Cette activité consiste à concevoir des contenus visuels combinant image et texte sous forme de vidéos, selon le cahier des charges du client.
- **Formations professionnelles** : YaneCode Academy dispense des formations certifiantes, garantissant un apprentissage de qualité.

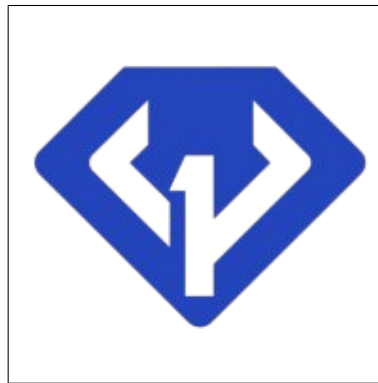


Figure 1.1 : :Logo YaneCode

L'écosystème YaneCode comprend également YaneCode Robotic et YaneCode Academy :

- **YaneCode Robotic** : est une boutique en ligne spécialisée dans la vente d'équipements robotiques (Arduino, Raspberry), offrant une gamme de produits adaptée aux besoins des clients[2].
- **YaneCode Academy** : est une plateforme de formations certifiantes en informatique développement web, mobile, design et robotique dispensées à Safi, visant l'insertion professionnelle des étudiants et diplômés[3].



Figure 1.2 : Logo YaneCode Robotique & Academy

1.2 Contexte général du projet

1.2.1 Problématique

Dans le domaine aéronautique, la surveillance des paramètres de vol et des systèmes embarqués nest pas un simple complément technique : cest le coeur de la sûreté opérationnelle. Un avion moderne embarque plusieurs centaines de capteurs qui, en temps réel, pilotent les commandes de vol, orchestrant les communications et contrôlant en permanence le comportement des moteurs. La moindre dérive, si elle nest pas détectée à temps, peut conduire à des situations critiques. Cest pourquoi la fiabilité des équipements et des logiciels de monitoring est placée au plus haut niveau des exigences de conception et de maintenance.

Malgré leur nécessité, les systèmes de surveillance déployés présentent plusieurs lacunes récurrentes Ces limites peuvent être regroupées autour de sept axes principaux :

- **Absence de centralisation** : Les données sont dispersées entre des systèmes hétérogènes, ce qui empêche toute vision unifiée de l'état des systèmes critiques.
- **Collecte difficile** : Les capteurs évoluent dans des environnements extrêmes, avec des contraintes sévères de consommation et de fiabilité qui compliquent l'acquisition des données.
- **Visualisation insuffisante** : Les interfaces cloisonnées et peu intuitives rendent difficile une interprétation rapide et claire de l'état des systèmes en temps réel.
- **Stockage restreint** : La mémoire embarquée est limitée, et les solutions de stockage externes imposent une dépendance au réseau qui n'est pas toujours viable.
- **Détection réactive** : Les approches traditionnelles, basées sur des seuils fixes, interviennent après coup sans permettre une anticipation locale des anomalies.
- **Coût et complexité** : Les solutions existantes sont onéreuses et complexes à déployer, ce qui les rend inaccessibles pour de nombreux opérateurs.
- **Dépendance réseau** : La plupart des systèmes nécessitent une connectivité permanente pour la transmission et l'analyse des données, créant ainsi une vulnérabilité critique en vol. Face à ces contraintes, la problématique centrale du projet peut être formulée comme suit :

Comment concevoir une solution IoT embarquée légère, fiable et économique,

capable d'assurer la collecte, le stockage, le traitement local et la visualisation des paramètres critiques aéronautiques, tout en intégrant une capacité de détection précoce des anomalies en temps réel, sans dépendance à une infrastructure réseau externe ?

1.3 Étude et critique de l'existant

Avant de concevoir notre propre système, prenons le temps d'examiner les solutions déjà disponibles. Cette analyse nous éclairera sur leurs atouts et leurs limites, et nous aidera à justifier le choix d'une architecture plus légère, plus autonome et mieux adaptée aux réalités du terrain aéronautique.

1.3.1 Analyse des solutions existantes de monitoring

L'étude de l'existant présentée dans l'annexe 2 permet de repérer plusieurs solutions dédiées au monitoring des systèmes embarqués, également à la maintenance prédictive et à la détection d'anomalies. Les solutions identifiées présentent des différents niveaux de complexité, allant de prototypes IoT à faible coût tel que le projet **Smart Aircraft Maintenance** jusqu'à des plateformes industrielles dédiées au domaine aéronautique tel que le projet **Honeywell Forge**. [7]

TinyML low-power edge device for predictive maintenance : Une première solution étudiée est un prototype de maintenance prédictive qui s'appuie sur un mécanisme Edge à faible consommation. Cette architecture s'appuie notamment sur une carte embarquée ESP32 associée à un accéléromètre ADXL355, une passerelle Raspberry Pi et le protocole MQTT. Les données sont traitées au niveau du dispositif embarqué afin de détecter les anomalies vibratoires, puis les résultats sont envoyés vers une plateforme Cloud pour leur visualisation. [9]

Honeywell Forge Une autre approche implique une plateforme industrielle de monitoring aéronautique comme Honeywell Forge. Cette solution s'appuie sur l'IoT, le Cloud et l'intelligence artificielle afin d'assurer le diagnostic prédictif de divers sous-systèmes aéronautiques. Elle permet notamment de promouvoir non seulement des diagnostics prédictifs, des alertes en temps réel mais aussi des indicateurs de fiabilité à l'échelle d'une flotte. Son principal intérêt se présente dans son approche

multi-systèmes et dans l'intégration de capacités d'intelligence artificielle au plus près des équipements. [7]

Détection d'anomalies vibratoires sur moteurs par TinyML Une troisième solution identifiée concerne sur la détection d'anomalies vibratoires sur des moteurs à l'aide d'une architecture IoT combinant un carte embarquée ESP32-S3, un capteur MPU6050, le protocole MQTT et le middleware Node-RED. cette approche combine une classification supervisée à une méthode de détection d'anomalies fondée sur le clustering. Elle réalise également une extraction de caractéristiques temporelles et de fréquences avant la détection.[8]

Enfin, l'état de l'art met en avant la structures des architectures de maintenance prédictive aéronautique combinant de plus les capteurs embarqués, Edge Computing et services Web Cloud. Ces architectures visent aussi à rapprocher le traitement de la source des données afin de réduire la latence et la bande passante, tout en conservant une infrastructure éloignée pour l'accès aux mesures de performances du système. Elles mettent notamment en évidence l'importance de la protection des échanges et de l'intégrité des données.

Ainsi, l'étude des solutions actuelles montre une évolution vers des architectures décentralisées combinant acquisition embarquée, traitement local, communication IoT, stockage distant et supervision. Ce choix est particulièrement adapté à l'objectif du présent projet, qui consiste à construire un prototype capable d'acquérir plusieurs paramètres, de les prétraiter, de détecter des comportements anormaux et de restituer l'état du système à travers une interface de monitoring.

1.3.2 Critique de l'existant

Cette solution expose l'intérêt d'utiliser des composants accessibles et une architecture proche de celle recherchée dans le présent projet. Elle démontre également la pertinence du traitement Edge pour réduire la quantité de données transmises. Toutefois, il s'agit d'un prototype de type *maker* dont la validation quantitative des performances de détection reste limitée. **tinyml_edge**

Cependant, cette solution repose sur un écosystème industriel propriétaire et sur des contraintes de certification qui la rendent difficilement transposable à un prototype

académique.

Son architecture démontre une forte proximité avec le périmètre du présent projet, notamment grâce à l'association entre traitement Edge, communication MQTT et supervision temps réel. Toutefois, sa validation reste principalement réalisée en laboratoire et se limite à un seul type de signal, à savoir les vibrations. **tinyml_vibration**

1.4 Présentation du cahier des charges

Sur la base des limites identifiées dans l'étude de l'existant, les objectifs du système à concevoir ont été définis. Nous allons à présent détailler le cahier des charges fonctionnel et technique, qui spécifie les exigences auxquelles la solution devra répondre.

1.4.1 Description du projet

Le projet consiste à développer un prototype de système IoT embarqué pour la surveillance des paramètres critiques dans les systèmes aéronautiques. Ce dispositif utilise des capteurs et des microcontrôleurs pour l'acquisition en temps réel des données de vol, et intègre des algorithmes de détection d'anomalies pour une analyse locale et immédiate. Une interface de visualisation permet aux opérateurs de superviser l'état du système de manière intuitive. Conçu pour fonctionner en totale autonomie, sans recours à une infrastructure réseau, il assure une surveillance continue et fiable.

1.4.2 Objectif du projet

Conformément au cahier des charges présente dans l'annexe 1, ce projet poursuit quatre objectifs principaux :

- **Concevoir une solution embarquée complète** : réaliser un prototype assurant l'acquisition, le traitement et le stockage local des données critiques .
- **Permettre une détection précoce des anomalies** : développer des algorithmes capables d'identifier en temps réel tout écart comportemental .
- **Faciliter la supervision** : créer une interface intuitive pour une visualisation

claire et réactive de l'état du système.

- **Garantir la continuité opérationnelle** : maîtriser la consommation énergétique et supprimer la dépendance réseau.

1.4.3 Résultats attendus

À l'issue du projet, les livrables suivants sont attendus :

- Un prototype opérationnel du système de surveillance embarquée ;
- Une acquisition fiable et en continu des paramètres aéronautiques ;
- Une capacité de détection automatique et immédiate des anomalies ;
- Une interface de supervision ergonomique et réactive ;
- Une solution autonome, sobre en énergie et non connectée ;
- Une campagne de tests validant le prototype en environnement simulé

1.5 Méthodologie adoptée

Afin de mener à bien la réalisation de ce projet, il est essentiel de s'appuyer sur une démarche structurée et adaptée à sa nature. Cette section présente la méthodologie retenue, justifie ce choix et décrit son application tout au long du développement.

1.5.1 Présentation de la méthodologie

Kanban est un framework Agile visuel plus flexible et moins prescriptif que Scrum qui organise le travail en colonnes sur un tableau, avec des cartes pour suivre chaque tâche [].

1.5.2 Justification du choix

Pour le développement de ce projet, nous avons opté pour la méthodologie Kanban. Ce choix a été motivé par plusieurs raisons :

- **Flexibilité** : Kanban permet d'ajuster les priorités en cours de projet sans perturber l'ensemble du processus, ce qui est particulièrement adapté à un projet de recherche et développement où certaines spécifications peuvent évoluer [4].
- **Visualisation du flux de travail** : le tableau Kanban offre une vue d'ensemble

claire des tâches à réaliser, en cours et terminées, facilitant le suivi et la coordination [5].

- **Gestion des goulots d'étranglement** : la méthode permet d'identifier rapidement les étapes bloquantes et d'y remédier efficacement [4].

1.6 Application des phases au projet

Le développement du projet a été organisé en plusieurs phases successives, chacune répondant à un objectif spécifique. L'avancement des travaux a été planifié et suivi selon les étapes suivantes.

- **État de l'art et cadrage du sujet (Semaine du 08/07/2026)**

Avant de passer à la phase de réalisation, il était essentiel de prendre le temps d'analyser les différentes possibilités. Nous avons comparé les cartes embarquées, notamment l'ESP32, le STM32 et le Raspberry Pi, étudié les capteurs disponibles et évalué les différents protocoles de communication.

L'objectif n'était pas simplement de sélectionner une solution, mais de justifier chaque choix en fonction des besoins du projet. Cette étape de réflexion nous a permis de définir une architecture cohérente et de poser des bases solides pour la suite du développement.

- **Choix de la carte, des capteurs et mise en place de l'environnement de développement (Semaine du 15/07/2026).**

Une fois les décisions arrêtées, place à l'action concrète. Nous avons préparé la carte sélectionnée, connecté les capteurs (ou mis en place une simulation réaliste en l'absence de matériel) et installé tous les outils requis. Les premiers tests de lecture brute des capteurs ont représenté une étape cruciale : enfin, des données issues de l'équipement !

- **Acquisition et filtrage des données (Semaine du 22/07/2026)** Une fois les choix techniques validés, nous sommes passés à la phase de mise en oeuvre. Nous avons configuré la carte embarquée retenue, connecté les différents capteurs ou, lorsque le matériel n'était pas disponible, mis en place une simulation réaliste. En parallèle, les outils nécessaires au développement ont été installés et configurés. Les premiers tests de lecture des données des

capteurs ont marqué une étape importante du projet : ils ont permis de vérifier le bon fonctionnement de l'ensemble et de confirmer que les premières données étaient correctement acquises.

— **Détection des anomalies (Semaine du 29/07/2026)**

C'est durant cette semaine que le projet a véritablement trouvé son sens. Nous avons mis en place des seuils adaptatifs qui peuvent se calibrer sur la moyenne mobile, puis exploré une première méthode d'apprentissage automatique léger. Le but était de différencier un comportement normal d'une anomalie, d'une montée abrupte ou d'une défaillance.

— **Transmission des données via MQTT et développement du backend (Semaine du 05/08/2026)**

Un système qui ne transmet pas d'informations est une boîte noire. Ainsi, nous avons relié l'élément embarqué au monde extérieur grâce à un broker MQTT, créé un backend minimaliste pour réceptionner et authentifier les messages et installé une base de données pour stocker les précédents échanges. Pour la première fois, il était possible de faire transiter une donnée du capteur jusqu'à l'espace de stockage.

— **Développement de l'interface de supervision et du système d'alertes (Semaine du 12/08/2026).**

Tout ce travail technique devrait être mis en valeur. Nous avons développé une interface web pour exposer les données en direct, et avons incorporé un mécanisme d'alerte visuelle pour indiquer les irrégularités. Cette interface est ce qui rendra la démonstration finale compréhensible et facile d'accès.

— **Tests, validation et documentation (Semaine du 19/08/2026)**

Un prototype qui fonctionne í sur mon appareil ž ne suffit pas. Nous avons mis en oeuvre et consigné trois scénarios de test : le scénario standard, le scénario extrême et le scénario anormal. Chaque essai a été documenté avec ses entrées, ses résultats ainsi que des images de l'écran. Nous avons aussi élaboré les manuels d'installation et de vérification.

— **Finalisation, déploiement et rédaction du rapport (Semaine du 23/08/2026)**

Finition de la mission : mise en oeuvre de la version définitive, préparation de la présentation, et élaboration du rapport concluant. Après une ultime vérification de la documentation, un nettoyage du dépôt, le projet était fin prêt à être livré.

1.7 Planification du projet

Le planning prévisionnel de notre projet a été structuré en cinq phases principales, reflétant les étapes de la méthodologie kanban adoptée. Chaque phase regroupe un ensemble de tâches planifiées sur une durée totale de huit semaines comme le montre la figure suivante.

Phase / Semaine	P1 02/07 - 08/07	P2 09/07 - 22/07	P3 23/07 - 29/07	P4 30/07 - 12/08	P5 13/08 - 23/08
Phase 1 — Cadrage et etude	P1 (02/07 - 08/07)				
S1 — Etat de l'art + cadrage du sujet	S1				
Phase 2 — Developpement embarque (carte, capteurs, acquisition)		P2 (09/07 - 22/07)			
S2 — Choix carte/capteurs + setup environnement		S2			
S3 — Acquisition et filtrage des donnees		S3			
Phase 3 — Detection d'anomalies			P3 (23/07 - 29/07)		
S4 — Detection d'anomalies (seuils / ML léger)			S4		
Phase 4 — Backend, transmission et interface				P4 (30/07 - 12/08)	
S5 — Transmission MQTT + backend + stockage				S5	
S6 — Interface de monitoring + alertes				S6	
Phase 5 — Tests, documentation et livraison					P5 (13/08 - 23/08)
S7 — Tests (nominal/limite/erreur) + documentation					S7
S8 — Finalisation, déploiement, demo, rapport					S8

Figure 1.3 : Acteur systeme

La planification et le suivi de lavancement du projet ont été réalisés à travers la plateforme mise à disposition par lorganisme daccueil, Yancode. Cette plateforme constitue un outil central de gestion et de suivi du travail réalisé tout au long du projet.

Elle permet notamment dassurer une livraison quotidienne (ñ Daily z), au cours de laquelle sont renseignées les tâches prévues et réalisées pour chaque journée, ainsi que la présence et la participation aux activités du projet. En complément, un rapport hebdomadaire est déposé chaque semaine afin de présenter létat davancement du projet, les tâches réalisées, les difficultés et problèmes rencontrés, ainsi que les besoins spécifiques identifiés au cours de la semaine.comme le montre la figure suivante.

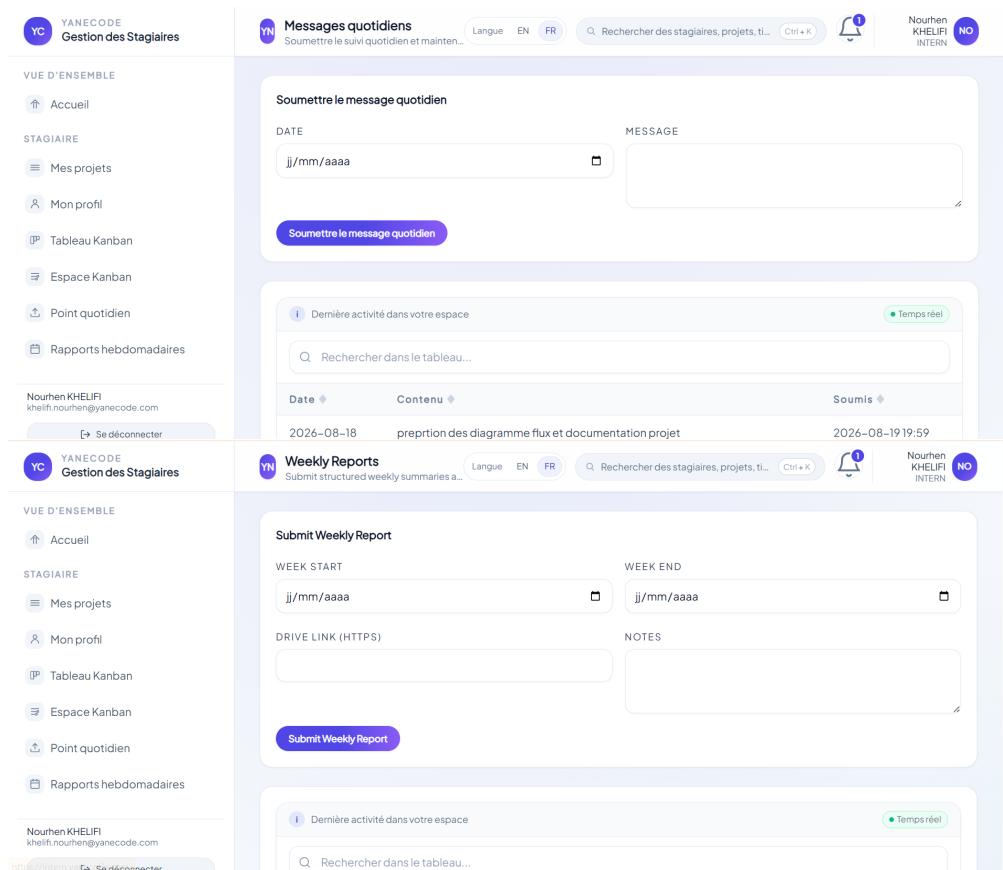


Figure 1.4 : Interfaces de suivi de lavancement du projet

Ce dispositif permet ainsi d'assurer une traçabilité continue de l'avancement, tout en facilitant la planification des tâches selon leur priorité, leur échéance (deadline) et leur état d'avancement. Il permet également de suivre l'atteinte des différents objectifs fixés et d'identifier rapidement les éventuels écarts ou blocages nécessitant une intervention ou une réorganisation des activités, comme l'indique la figure suivante .

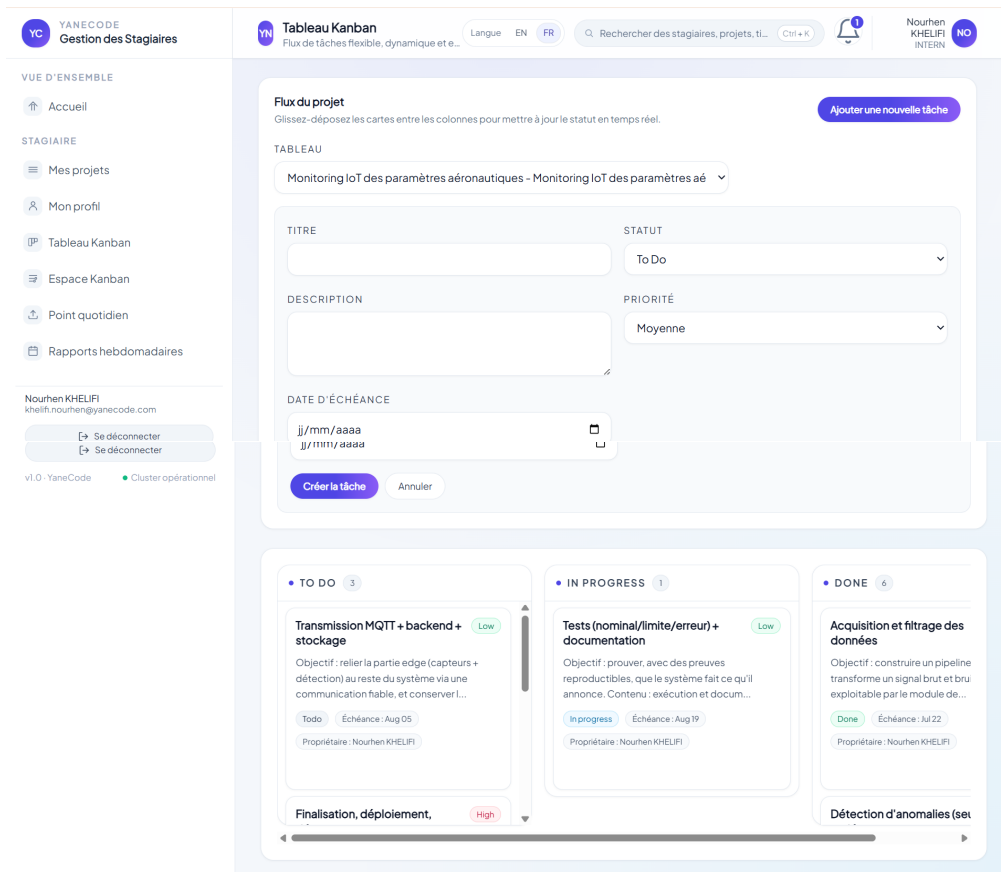


Figure 1.5 : Interfaces de planification du projet

conclusion

Ce chapitre a établi les bases du projet en détaillant le contexte général et la structure hôte. La question du suivi des paramètres aéronautiques a été soulevée, soulignant les insuffisances des systèmes actuels : collecte complexe, détection passive, coût exorbitant et dépendance à la connectivité réseau. L'examen approfondi des systèmes embarqués a validé l'efficacité d'une stratégie décentralisée qui allie le traitement Edge, la communication MQTT et la supervision en temps réel. Ainsi, les objectifs et les fonctionnalités attendues de la solution ont été spécifiés dans le cahier des charges, c'est-à-dire un prototype autonome, une détection précoce des anomalies et une interface conviviale. On a choisi d'adopter la méthode Kanban en raison de sa flexibilité et le calendrier prévu a organisé le développement sur une période de huit semaines. Le chapitre suivant sera consacré à l'état de l'art des technologies clés du projet.

État de l'art et justification des choix technologiques

Introduction

Dans ce chapitre, nous dressons un état de l’art des technologies et composants clés qui constituent la base de notre solution de surveillance embarquée.

Nous commençons par explorer les architectures IoT et les systèmes embarqués pour le monitoring, en comparant trois modèles distincts : centralisé, Edge Computing et distribué EdgeFogCloud afin de justifier notre choix d’une architecture décentralisée adaptée aux contraintes aéronautiques. Nous analysons ensuite les principales plateformes matérielles disponibles (ESP32, STM32 et Raspberry Pi), en évaluant leurs performances, leur consommation et leur adéquation avec les exigences du projet. Les capteurs susceptibles de mesurer les paramètres critiques : température, pression, vibration et mouvement sont également étudiés et comparés. Nous abordons enfin les techniques d’acquisition, de filtrage et de détection d’anomalies, ainsi que les protocoles de communication, les bases de données temporelles et les outils de supervision envisagés pour la chaîne de traitement complète.

2.1 Systèmes IoT et architectures embarquées pour le monitoring

2.1.1 Définition de l’IoT

L’Internet des Objets (IoT) désigne l’ensemble des objets physiques connectés à Internet, capables de collecter et d’échanger des données. Dans le contexte industriel, on parle d’IIoT (Industrial Internet of Things) pour désigner les applications dans les environnements critiques.

2.1.2 Architecture IoT

Une architecture IoT typique se compose de plusieurs couches :

- **Couche perception** : Capteurs et actionneurs
- **Couche réseau** : Protocoles de communication
- **Couche middleware** : Traitement et gestion des données
- **Couche application** : Interfaces utilisateur et services

2.1.3 Systèmes embarqués dans les applications critiques

Les systèmes embarqués sont des systèmes informatiques intégrés à des dispositifs physiques. Dans les applications critiques, ils doivent répondre à des exigences strictes de fiabilité, de temps réel et de sécurité.

2.1.4 Analyse des architectures embarquées existantes

L'analyse des solutions étudiées permet de révéler trois architectures distincts selon l'emplacement des fonctions liées au traitement des données et la prise de décision.

Architecture centralisée : La première architecture correspond à une organisation de type **Edge–Gateway–Cloud**. Les capteurs transmettent l'ensemble des données brutes vers une unité de traitement centrale, généralement située dans le cloud. Celle-ci prend en charge toutes les opérations : filtrage, analyse, et détection d'anomalies. Ce modèle simplifie la gestion globale du système et permet une vue unifiée, mais il induit une forte dépendance vis-à-vis de la connectivité réseau et de l'unité centrale, avec des latences plus élevées et un risque de saturation de la bande passante. [10]

Architecture Edge Computing : Une deuxième architecture correspond à une approche **Edge–MQTT–Monitoring**. À l'inverse, l'architecture Edge Computing déplace une partie significative du traitement au plus près des capteurs, directement sur le dispositif embarqué ou sur une passerelle locale. Les données y sont filtrées, prétraitées et parfois analysées localement, ce qui permet de réduire le volume de données transmises, de diminuer la latence et de maintenir certaines fonctions de surveillance même en cas d'indisponibilité temporaire du réseau. Cette approche est particulièrement adaptée aux opérations critiques en temps réel et à la détection locale d'anomalies.

Architecture distribuée EdgeFogCloud :

Une troisième architecture, plus orientée vers les applications industrielles aéronautiques largement documentée pour les systèmes IoT/IIoT, repose sur une architecture distribuée EdgeFogCloud. Les traitements sont répartis sur plusieurs niveaux :

La couche Edge assure l'acquisition des données et les traitements nécessitant une très faible latence (filtrage, règles simples, détection basique).

La couche intermédiaire (Fog) prend en charge l'orchestration, l'agrégation, et la transmission des données, ainsi que des analyses régionales ou une inférence ML légère.

La couche Cloud (ou serveur) assure principalement la persistance des données, les analyses approfondies, l'entraînement de modèles et la visualisation.

Cette architecture est particulièrement adaptée au contexte de votre projet.

Le dispositif embarqué peut effectuer localement l'acquisition des données, leur filtrage et une première détection basée sur des règles. Les données résultantes peuvent ensuite être transmises par MQTT vers une couche middleware basée sur Node-RED, qui joue le rôle d'intermédiaire (Fog) entre le système embarqué et les services de stockage et de visualisation.

Les données temporelles peuvent être enregistrées dans une base de données adaptée (par exemple une base temporelle), puis exploitées graphiquement via une plateforme de supervision telle que Grafana. [12] Ainsi, l'étude comparative des architectures étudiées montre donc que préférence pour la répartition des traitements au lieu de leur centralisation. Les fonctions nécessitant une réponse rapide peuvent être réalisées au niveau Edge, alors que les fonctions de conservation historique, d'affichage et d'analyse plus complexe peuvent être déportées vers des couches plus élevées.

Dans le cadre de notre projet, cette organisation nous amène à utiliser une architecture en plusieurs niveaux. La couche Edge est chargée de l'obtention et le préchargement des données, en plus de la détection à l'échelle locale.

Une couche d'intermédiation gère la transmission et l'orchestration des données via MQTT. Finalement, la couche dédiée à la persistance et à la présentation des données sert à conserver l'historique et à observer l'évolution des paramètres surveillés.

La structure est aussi conforme à l'architecture cible spécifiée dans le cahier des charges, laquelle stipule que les capteurs fournissent les données à un système embarqué qui opère le traitement Edge, avant l'envoi des informations

vers un backend, une base de données et une interface de surveillance.

2.2 Cartes embarquées et plateformes de traitement

2.2.1 ESP32

La puce SoC (System on a Chip) de l'ESP32 (DevKit C v4) d'Espressif Systems est très populaire dans le domaine de l'IoT. Cet appareil est doté d'un processeur dual-core Xtensa LX6 tournant à 240 MHz, avec 520 KB de mémoire vive, et il inclut de façon native des modules WiFi 802.11 b/g/n ainsi que Bluetooth v4.2 (BLE et Classic). Avec son prix raisonnable (environ 5) et son incorporation directe dans des simulateurs tels que Wokwi, c'est une option privilégiée pour les projets nécessitant un prototypage rapide.[17] De plus, l'ESP32 dispose de 34 broches GPIO, d'interfaces I2C, SPI, UART et de convertisseurs ADC/DAC, lui conférant une grande flexibilité pour la collecte de données à partir de plusieurs capteurs. Sa consommation d'énergie est optimisée grâce à l'utilisation de modes veille approfondis (Deep Sleep), permettant de réduire la consommation à moins de 10 μ A. [18]

2.2.2 STM32

Le groupe STM32 (STMicroelectronics) est centré sur les coeurs ARM Cortex-M, tels que le M0+, le M3, le M4 et le M7. Elle propose une solidité industrielle, une large gamme de périphériques et une gestion efficace de la consommation, spécifiquement pour les séries ultra-low-power (STM32L4, STM32L5). Toutefois, les modèles de rang moyen et d'entrée n'incluent généralement pas la connectivité sans fil de manière native, ce qui nécessite des modules externes supplémentaires (ESP-01, HC-05, etc.). Cette situation accroît la complexité du PCB et le coût global.[19]

Les STM32 sont souvent choisis pour les applications en temps réel strictes (comme le contrôle de moteur ou les systèmes de freinage) en raison de leur fiabilité en termes de déterminisme et de leur compatibilité avec les RTOS tels que FreeRTOS et Zephyr.[20]

2.2.3 Raspberry Pi

Les mini-ordinateurs Raspberry Pi, comme le Zero W ou le 4, fournissent des performances de traitement comparables à celles d'un ordinateur traditionnel grâce à l'utilisation de systèmes d'exploitation basés sur Linux (Raspberry Pi OS, Ubuntu). Bien qu'ils soient parfaits pour des algorithmes intensifs (comme le traitement d'image ou l'inférence de modèles de deep learning), leur consommation en énergie (100500 mA), leur durée de démarrage (3060 s) et l'absence de garantie de temps réel strict les rendent moins appropriés pour un élément de capteur intégré de bas niveau. [21]

2.2.4 Comparaison des plateformes

Critère	ESP32	STM32	RPi
Performance	Moyenne	Élevée	Très élevée
Consommation	Très faible	Faible	Élevée
Connectivité	Intégrée	Externe	Intégrée
Coût	\$5-10	\$10-50	\$35-100
Facilité d'utilisation	Élevée	Moyenne	Élevée
Edge AI	Limité	Bonne	Excellente
Temps réel	Bon	Excellent	Limité

Tableau 2.1 : Comparaison des plateformes embarquées

2.2.5 Justification du choix de la plateforme

Pour ce projet, le choix se porte sur l'ESP32 pour les raisons suivantes :

1. **Connectivité intégrée** : Le Wi-Fi et le Bluetooth intégrés simplifient la communication sans modules supplémentaires.
2. **Coût accessible** : Permet un prototype économique et reproductible.
3. **Faible consommation** : Adapté à une utilisation autonome.
4. **Support Edge AI** : Compatible avec TensorFlow Lite pour la détection d'anomalies.

5. **Facilité de développement** : Environnement Arduino familier et grande communauté.
6. **Capacités suffisantes** : Puissance de calcul adaptée aux algorithmes de détection choisis.

2.3 Capteurs et paramètres critiques aéronautiques

2.3.1 Température

La température est un paramètre critique dans les systèmes aéronautiques.

Importance :

- Surveillance des moteurs pour éviter la surchauffe
- Gestion de la température de la cabine
- Détection des composants défaillants
- Sécurité des équipements électroniques

Capteurs envisagés :

- DS18B20 : Capteur numérique 1-Wire, précision $\pm 0.5^{\circ}\text{C}$
- LM35 : Capteur analogique, précision $\pm 0.5^{\circ}\text{C}$
- Thermocouple type K : Haute température (jusqu'à 1250°C)
- DHT22 : Température et humidité

2.3.2 Pression

La pression est un paramètre essentiel pour les systèmes de contrôle de vol et de pressurisation.

Importance :

- Contrôle de l'altitude et de la vitesse
- Surveillance de la pression des cabines
- Détection des fuites hydrauliques
- Contrôle des systèmes de carburant

Capteurs envisagés :

- BMP180 : Pression et température, précision ± 0.01 hPa
- BMP280 : Version améliorée du BMP180
- MS5611 : Haute précision pour l'altitude

- MPXV7002 : Capteur de pression différentielle

2.3.3 Vibration et mouvement

Les vibrations et les mouvements sont des indicateurs clés de l'état mécanique des aéronefs.

Importance :

- Détection des déséquilibres rotoriques
- Surveillance de l'usure des paliers
- Analyse des mouvements structurels
- Détection des impacts ou turbulences

Capteurs envisagés :

- MPU6050 : Accéléromètre 3 axes + Gyroscope 3 axes
- ADXL345 : Accéléromètre haute résolution
- SW-420 : Capteur de vibration
- LIS3DH : Accéléromètre 3 axes avec détection de chute

2.3.4 Comparaison des capteurs étudiés

Capteur	Paramètre mesuré	Précision / Résolution	Interface
DS18B20	Température	±0,5 °C	1-Wire
BMP280	Pression atmosphérique	±0,01 hPa	I2C / SPI
MPU6050	Accélération / Gyroscope	Acc. : ±2g à ±16g (16 bits)	I2C
DHT22	Température + Humidité	±0,5 °C, ±2 % HR	1-Wire

Tableau 2.2 : Comparaison des capteurs environnementaux

2.3.5 Justification des paramètres surveillés

Les paramètres suivants sont retenus pour ce projet :

- **Température** : Pour simuler la surveillance des moteurs et des composants

critiques.

- **Pression** : Pour simuler les variations liées à l'altitude et à la pressurisation.
- **Vibration** : Pour simuler la détection des anomalies mécaniques.
- **Mouvement** : Pour simuler l'attitude et les mouvements de l'aéronef.

Ces paramètres sont représentatifs des systèmes de surveillance aéronautique tout en restant simulables avec des capteurs abordables et accessibles.

2.4 Acquisition et prétraitement des données

2.4.1 Acquisition des données capteurs

L'acquisition des données est réalisée via les interfaces I2C et 1-Wire de l'ESP32. Les données sont échantillonnées à une fréquence adaptée (typiquement 10 Hz).

2.4.2 Bruit et perturbations des mesures

Les mesures issues des capteurs sont sujettes à différentes sources de bruit :

- Bruit thermique
- Bruit d'échantillonnage
- Interférences électromagnétiques
- Bruit de quantification (ADC)

2.4.3 Techniques de filtrage

Afin d'assurer l'excellence des données collectées, on applique un broyage de proximité sur les signaux non traités, qui sont particulièrement sensibles au bruit (en particulier l'accéléromètre). Nous avons mis en place un filtre de moyenne mobile sur l'ESP32 qui utilise une taille de fenêtre de 5. L'équation de filtrage se formule de la manière suivante :

$$y_k = \frac{1}{N} \sum_{i=0}^{N-1} x_{k-i} \quad (2.1)$$

$y[n]$ est la valeur filtrée, $x[n]$ sont les valeurs acquises précédemment, et $N=5$. Ce lissage évite les déclenchements de fausses alertes dues à des pics

de mesure aberrants, tout en conservant une latence minimale (quelques millisecondes).[21]

2.4.3.1 Filtre moyenne glissante

Ce filtre consiste à calculer la moyenne des N dernières mesures.

Avantages : Simple à implémenter, efficace pour les bruits aléatoires.

Inconvénients : Nécessite une mémoire tampon, réponse lente.

2.4.3.2 Filtre exponentiel

Ce filtre pondère les valeurs récentes plus fortement que les anciennes.

$$y_k = \alpha \cdot x_k + (1 - \alpha) \cdot y_{k-1} \quad (2.2)$$

où α est la constante de filtrage ($0 < \alpha < 1$).

Avantages : Faible mémoire, temps de réponse ajustable.

Inconvénients : Réponse exponentielle, non-linéaire.

2.4.3.3 Filtre de Kalman

Le filtre de Kalman est une méthode de filtrage récursif qui estime l'état d'un système linéaire à partir de mesures bruitées.

$$\text{Prediction : } \hat{x}_{k|k-1} = A\hat{x}_{k-1|k-1} + Bu_k \quad (2.3)$$

$$P_{k|k-1} = AP_{k-1|k-1}A^T + Q \quad (2.4)$$

$$\text{Mise à jour : } K_k = P_{k|k-1}H^T(H P_{k|k-1}H^T + R)^{-1} \quad (2.5)$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(z_k - H\hat{x}_{k|k-1}) \quad (2.6)$$

$$P_{k|k} = (I - K_kH)P_{k|k-1} \quad (2.7)$$

Avantages : Estimation optimale, adaptatif.

Inconvénients : Complexité de mise en oeuvre, nécessite un modèle du système.

2.4.4 Justification du choix du filtrage

Pour ce projet, le filtre moyenne glissante est retenu pour les raisons suivantes :

- Simplicité d'implémentation sur l'ESP32
- Efficacité suffisante pour le prototype
- Ressources mémoire limitées
- Convient aux données échantillonnées à fréquence régulière

2.5 Détection d'anomalies et IA en Edge

Approche par Seuils : Threshold-based La méthode première mise en œuvre sur l'ESP32 est déterministe, basée sur des seuils de criticité :

- **Température** $> 45^{\circ}\text{C}$: risque de surchauffe
- **Pression** < 850 hPa : dépressurisation ou altitude critique
- **Vibrations** > 15 m/s² : défaillance mécanique imminente
- **Potentiomètre** > 80 pourcent : simulant un emballement ou une défaillance moteur "ENGINE FAILURE" [23] Le dépassement de l'un de ces seuils active instantanément l'actionneur d'alerte locale (Buzzer sur GPIO25 et LED Rouge sur GPIO27), garantissant une réactivité temps réel absolue, indépendante de la latence réseau.

2.5.1 Méthodes de Machine Learning étudiées

Afin d'envisager une détection d'anomalies plus complexe (anomalies contextuelles ou dérives lentes), plusieurs modèles ont été étudiés théoriquement pour une potentielle implémentation Cloud ou Edge ML :

- **Isolation Forest** : Efficace pour séparer les anomalies des données normales dans un espace multidimensionnel, adapté aux jeux de données massifs. Complexité : $(\log n) O(n \log n)$.
- **One-Class SVM** : Modélise le comportement normal et détecte toute déviation, utile lorsque seules des données de vol "normales" sont disponibles à l'entraînement. Complexité : $(n^2) O(n^2)$ à $(n^3) O(n^3)$.
- **Autoencoder LSTM** : Réseaux de neurones profonds excellant dans la détection d'anomalies sur des séries temporelles séquentielles (Time Series).

Nécessite un entraînement supervisé et des ressources GPU/TPU.

2.5.2 Comparaison des méthodes d'IA et Datasets

Ces modèles ont été mis en perspective en étudiant les bases de données SurveilDrone-Net23 (comportant plus de 140 256 enregistrements de vol de drones) et NASA SMAP/MSL Telemanom (données de télémétrie spatiale). L'Autoencoder LSTM montre une précision supérieure sur les séries temporelles (F1-Score 0.800.85), mais nécessite des ressources de calcul élevées, incompatibles avec un ESP32 sans optimisation drastique (quantification TensorFlow Lite).

Des études récentes montrent que l'Isolation Forest offre un meilleur compromis précision/latence pour les systèmes embarqués (F1-Score 0.75, latence < 1 ms).

2.5.3 Justification

Pour la réalisation pratique de ce PFA, l'approche par seuils statiques embarqués a été justifiée et implémentée car elle garantit le déterminisme, consomme très peu de ressources CPU/RAM (< 10 KB) et répond au besoin de réactivité instantanée pour des alertes critiques telles que la panne moteur.

2.6 Protocoles de communication IoT

2.6.1 Comparaison des protocoles (WiFi, Bluetooth, UART, MQTT)

Dans l'IoT, la transmission d'informations nécessite un protocole léger. L'UART et le Bluetooth (BLE) sont limités par leur portée physique (< 100 m). Pour la transmission réseau, des protocoles applicatifs tels que HTTP, WebSocket, CoAP et MQTT s'affrontent. Le Tableau suivant démontre la supériorité de MQTT pour les environnements IoT contraints. Son architecture de publication/souscription et son en-tête réduit à 2 octets minimisent la bande passante.

Tableau 2.3 : Comparaison des protocoles de communication IoT

Caractéristique	MQTT	HTTP/REST	CoAP	WebSoc
Architecture	Publish/Subscribe	Request/Response	Request/Response	Full-dup
Overhead (En-tête)	2 octets (min)	Très lourd (200800 octets)	Léger (~4 octets)	Moyen (3)
QoS (Qualité de Service)	Oui (0, 1, 2)	Non	Non	Non
Maintien de connexion	Automatique, robuste	Non (stateless)	UDP (non fiable)	Connexion

2.6.2 Justification du choix MQTT

Le protocole MQTT a été retenu. L'ESP32 publie les trames JSON sur le topic `aircraft/sensors` via le broker public `broker.hivemq.com` sur le port 1883. Ce choix assure un couplage lâche, une reconnexion automatique et une scalabilité optimale si l'on souhaitait ajouter des centaines de capteurs simultanément.

2.6.3 Justification du choix MQTT

Pour exploiter les données MQTT, un middleware (Node-RED) est utilisé pour router l'information vers la base de données, suivie d'un outil de visualisation.

- **Bases de données temporelles :** InfluxDB 3, TimescaleDB, Prometheus, SQLite.
- **Outils de visualisation :** Grafana, Kibana, Tableau, Power BI.
- **Simulateurs matériels :** Wokwi, Proteus, SimulIDE, TinkerCAD.

Tableau 2.4 : Sélection de l'écosystème logiciel

Composant	Solution Choisie	Justification
Simulateur	Wokwi	Support natif et gratuit de l'ESP32, simulation WiFi réelle, partage de projets simplifié par rapport à Proteus ou TinkerCAD (limité à Arduino).
Routage	Node-RED	Programmation visuelle, nuds MQTT in/out natifs, intégration facile avec les bases de données.
Stockage	InfluxDB 3 Core v3.11.0	Conçu spécifiquement pour les séries temporelles (Time Series). Création d'un bucket <i>aircraft</i> et d'une mesure <i>sensors</i> . Excellente compression par rapport à SQLite.
Visualisation	Grafana Cloud	Création d'un dashboard de 10 panneaux de contrôle. Intégration native avec InfluxDB, rafraîchissement performant configuré à 5 secondes, alertes visuelles personnalisables.

conclusion

Ce chapitre a permis d'explorer l'état de l'art des composants entrant dans la chaîne de valeur IoT et d'argumenter nos choix technologiques. L'architecture globale retenue s'articule autour d'un ESP32 simulé sous Wokwi, effectuant une acquisition lissée par moyenne mobile et une détection d'anomalies par seuils. La remontée des données est assurée par le protocole léger MQTT vers un broker HiveMQ, avant d'être orchestrée par Node-RED, stockée efficacement dans InfluxDB 3 et restituée dynamiquement sur Grafana. Le chapitre suivant sera consacré à la mise en uvre pratique et au déploiement de ces différentes briques logicielles.

Analyse des besoins et conception de la solution

3.1 Introduction

Ce chapitre constitue le coeur de la phase de conception du projet. Après avoir défini le contexte et justifié les choix technologiques dans les chapitres précédents, nous procédons dans ce chapitre à une analyse approfondie des besoins du système. Cette analyse permet d'identifier clairement les fonctionnalités attendues ainsi que les contraintes à respecter.

Nous présentons ensuite l'architecture détaillée de la solution, en distinguant les composantes matérielles et logicielles, avant de décrire le flux de données qui traverse l'ensemble du système. La conception est formalisée à l'aide des diagrammes UML, conformément aux bonnes pratiques du génie logiciel.

3.2 Identification des besoins

L'identification des besoins constitue une étape fondamentale dans la conception d'un système. Elle permet de définir clairement ce que le système doit faire (besoins fonctionnels) et comment il doit le faire (besoins non fonctionnels).

3.2.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les services que le système doit fournir et les fonctionnalités qu'il doit offrir. Pour notre projet, ils sont définis comme suit :

- **Acquisition** : Le système doit acquérir les données environnementales et cinématiques tel que température, humidité, pression, altitude, et accélération selon les 3 axes via les capteurs embarqués.
- **Simulation** : Le système doit permettre la simulation de pannes et d'états critiques .
- **Filtrage** : Le microcontrôleur doit être capable d'appliquer un filtrage temporel (moyenne mobile) pour lisser les données brutes des capteurs.
- **Détection embarqué** : Le système doit détecter localement des anomalies critiques et les déclencher en se basant sur des seuils de tolérance.
- **Transmission** : Les données doivent être formatées en JSON et transmises

vers un broker via le protocole MQTT.

- **Routing et transformation** : le flux de données doit être parsé, validé et préparé pour le stockage à travers un middleware.
- **Stockage** : Le système doit stocker les séries temporelles de manière persistante et structurée pour avoir des requêtes analytiques.
- **Visualisation** : le système doit prévoir Une interface graphique qui affiche les données en temps réel et l'historique des alertes.
- **Explicabilité ML** : Bien que non embarqués, les modèles de Machine Learning utilisés dans les projets entraînés sur les datasets choisis doivent pouvoir ingérer ces données pour des diagnostics complexes.

3.2.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent les qualités et les contraintes du système, indépendamment de ses fonctionnalités spécifiques.

- **Fiabilité** : Le système doit garantir l'exactitude des données collectées sans perte de paquets lors de la transmission MQTT .
- **Faible latence** : Les détections d'anomalies embarquées doivent se faire en quasi-temps réel .
- **Robustesse** : Le firmware ne doit pas se bloquer en cas de défaillance d'un capteur.
- **Maintenabilité et Modularité** : Le code doit être structuré en fonctions indépendantes, et l'architecture doit être découplée .
- **Scalabilité** : La solution cloud doit pouvoir intégrer de nouveaux nœuds capteurs sans refonte de l'infrastructure.
- **Faible consommation** : L'optimisation des intervalles de lecture et d'envoi contribue à l'efficacité énergétique du dispositif embarqué.

3.3 Identification des acteurs du système

L'identification des acteurs permet de comprendre les interactions entre les différentes entités du système. Chaque acteur joue un rôle spécifique dans le fonctionnement global.

Notre projet présente six acteurs, décrits dans le tableau suivant.

Acteur	Type	Rôle
Système embarqué (ESP32)	Système (Primaire)	Collecte les mesures physiques, applique un premier filtrage, génère des alertes locales et publie les données JSON sur le réseau.
Broker MQTT (HiveMQ)	Système (Intermédiaire)	Reçoit et distribue les messages (Publish/Subscribe) de manière asynchrone.
Middleware (Node-RED)	Système (Intermédiaire)	S'abonne aux données, parse les payloads JSON, et formate les requêtes d'insertion.
Système de stockage (InfluxDB)	Système (Secondaire)	Base de données Time Series assurant la persistance et l'interrogation rapide des historiques.
Opérateur de maintenance	Humain (Primaire)	Consulte le dashboard en temps réel, surveille les alertes, analyse le Risk Score et les explications XAI pour prendre des décisions.
Module ML / XAI	Système (Autonome)	S'abonne au flux de données, détecte les anomalies, génère le Risk Score et produit les valeurs SHAP explicatives.

Tableau 3.1 : Identification des acteurs du système

La figure suivante détaille les différents acteurs ainsi que leurs interactions avec le système.

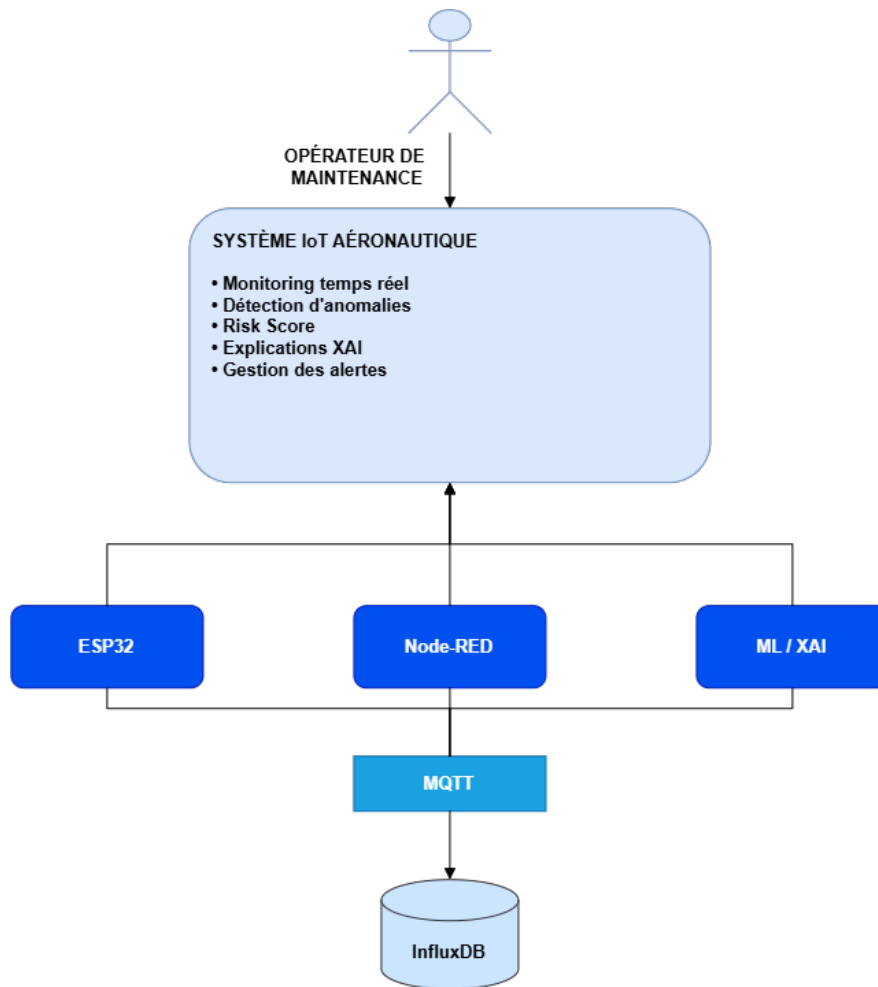


Figure 3.1 : Acteur systeme

3.4 Architecture détaillée du système

3.4.1 Architecture matérielle

L'architecture matérielle du système se compose de plusieurs éléments interconnectés. La figure suivante présente le câblage des différents composants

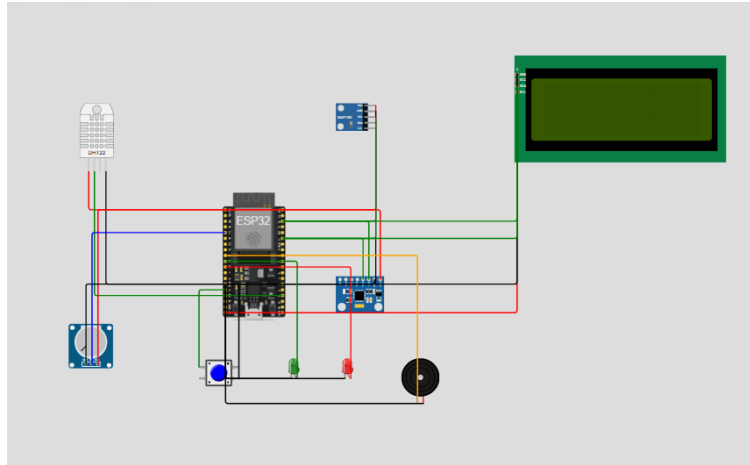


Figure 3.2 : Acteur systeme

Le tableau suivant présente l'interfaçage ainsi que la fonction de chacun des composants du système.

Périphérique	Interface / Pin	Fonction
ESP32 (Microcontrôleur)	-	Connectivité Wi-Fi intégrée, dual-core 240 MHz, traitement embarqué.
Capteur DHT22	GPIO15	Mesure de la température et de l'humidité.
Capteur BMP180	I2C (0x77)	Mesure de la pression atmosphérique et calcul de l'altitude.
Capteur MPU6050	I2C (0x68)	Centrale inertielle : accélération (X, Y, Z) pour l'analyse vibratoire.
Potentiomètre	GPIO34	Injecteur de fautes : simulation d'usure ou de défaillance des moteurs.
Écran LCD 20x4	I2C	Affichage des informations en local.
LEDs d'état	GPIO	Verte = fonctionnement normal, Rouge = alerte.
Buzzer	GPIO	Alarme sonore en cas de détection d'anomalie.

Tableau 3.2 : Architecture matérielle et périphériques associés à l'ESP32

3.4.2 Architecture logicielle

Nous avons opté pour une architecture distribuée a fin de séparer les fonctions dacquisition, de traitement local, de communication, de stockage, de visualisation et danalyse intelligente.En effet cette division suit une logique EdgeMiddlewareCloud, dans laquelle chaque niveau assure un rôle spécifique dans la chaîne de monitoring de notre système.

Lobjectif de cette architecture est de s'aligner avec larchitecture cible définie dans le cahier des charges a savoir transformer les mesures issues des capteurs en informations exploitables pour la surveillance de létat du système. elle est organisée en quatre couches distinctes et complémentaires. Cette structure garantit la modularité et l'extensibilité de la plateforme :

1. **Couche Edge** : Cette couche représente le premier niveau de l'architecture elle est exécutée au niveau de la carte embarquée , dont son role est principalement d'acquérir les paramètres physiques surveillés dans des intervalles réguliers et les filtrer avant leur transmission.

Dans cette couche on a choisi C++ avec Arduino Core comme un firmware avec une gestion temporelle en utilisant millis() pour exécuter périodiquement différentes tâches sans bloquer lexécution principale du programme avec des temporisations longues de type delay().

2. **Couche Middleware** : Cette couche assure la fonction de passerelle logicielle entre le systeme embarqué et la couche stockages ,elle prend en charge non seulement la reception des messages publiés par la carte embarqué via MQTT,mais également la vérification de leur structure et la transformation des données dans un format compatible avec la base de données.

3. **Couche de persistance** : Cette couche dédié à la conservation des données temporelles produites par le système embarqué.Les données regues et traitées par le middelware sont transmises vers la base de données afin dy être enregistrées sous forme de séries temporelles. Cette séparation permet de dissocier le traitement et lacheminement des données de leur stockage à long terme.

4. **Couche de présentation** : cette couche est destinée à l'exploitation visuelle des données de monitoring à travers Grafana qui se connecte à la couche persistance dans l'ordre d'exploiter et de représenter graphiquement les données stockées dans la base de données.

la Figure suivante illustre l'organisation de ces quatre couches ainsi que leurs interactions.

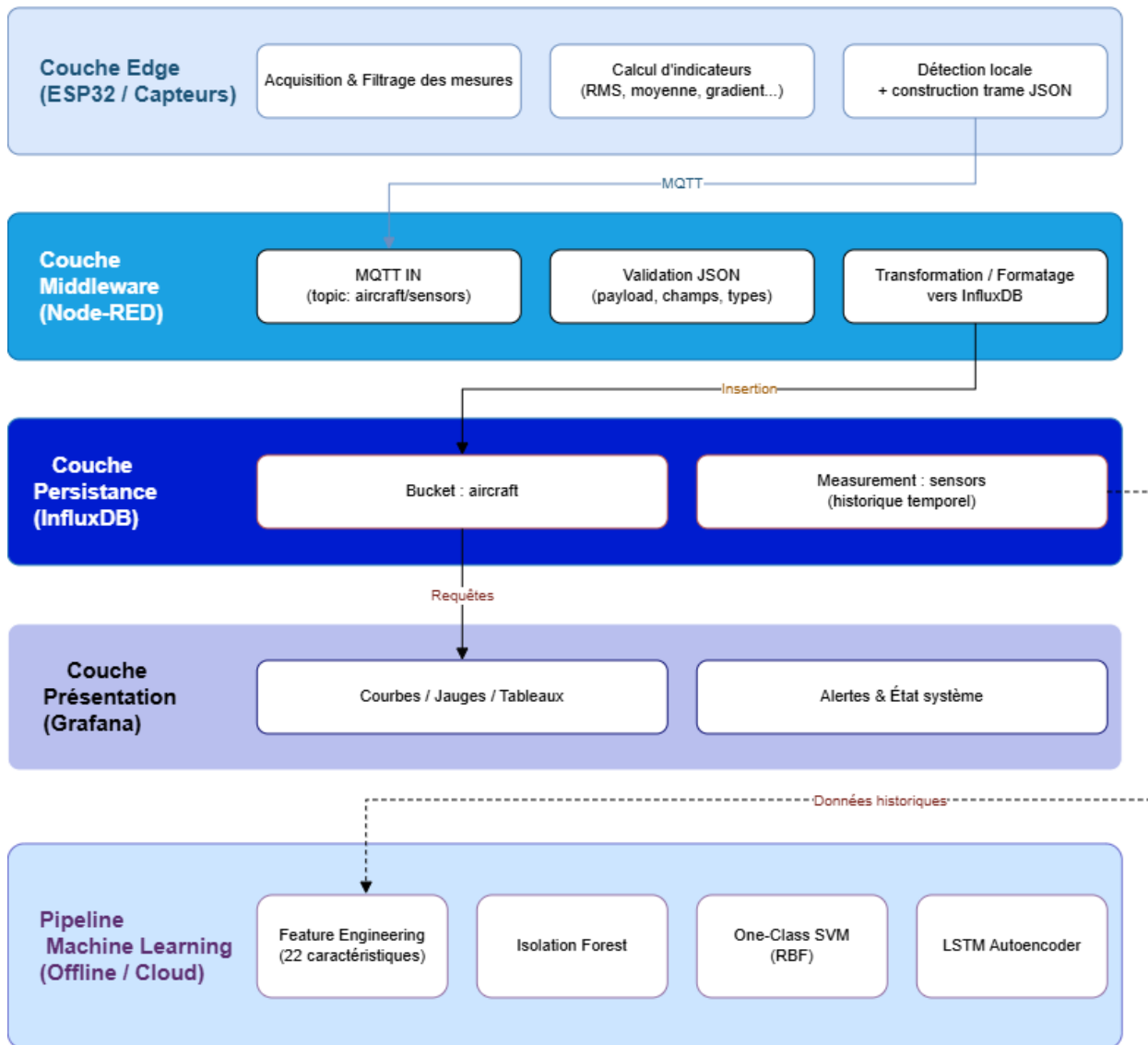


Figure 3.3 : edge flux de données

3.4.3 Flux de données

Les données sont d'abord acquises au niveau embarqué, puis prétraitées localement, transmises par le protocole MQTT, en fin stockées afin de permettre

leur visualisation et leur analyse historique comme le montre la figure suivante

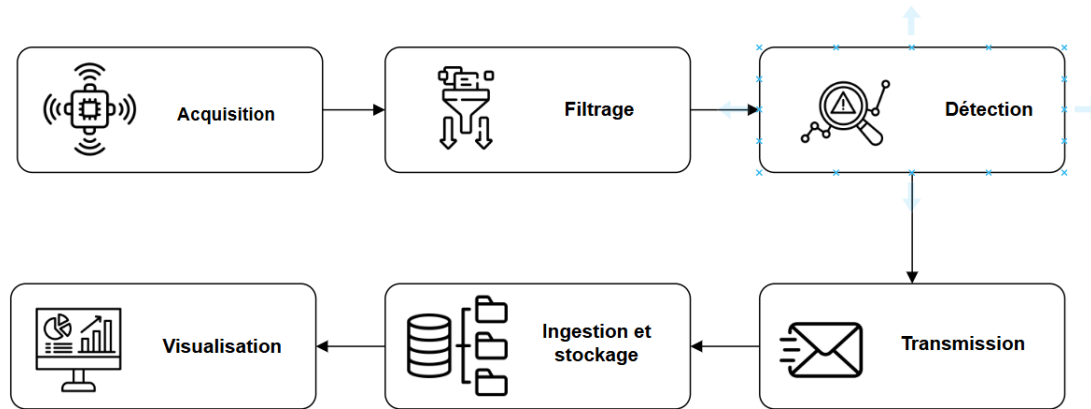


Figure 3.4 : edge flux de données

Description détaillée du flux de données

1. **Acquisition** : Les capteurs sont lus à intervalle régulier toutes les 1000 ms. Les données brutes sont ensuite converties en valeurs physiques telles que température en °C, pression en hPa, etc.
2. **Filtrage** : Les valeurs acquises sont ajoutées puis traitées au buffer d'un filtre à moyenne mobile. La valeur filtrée est calculée en faisant la moyenne des N dernières mesures pour réduire les fluctuations et le bruit de mesure.
3. **Détection** : La valeur filtrée est comparée aux seuils configurés pour chaque paramètre surveillé. une anomalie est détectée et un événement est généré Lorsqu'une valeur dépasse un seuil.
4. **Transmission** : Les données (brutes, filtrées, état de l'anomalie) sont formatées en JSON et publiées sur le broker MQTT toutes les 3000 ms.
5. **Stockage** : Les données reçues et traitées par Node-RED, sont écrites dans un fichier de log pour l'historique.
6. **Visualisation** : L'interface de monitoring affiche en temps réel les données reçues via MQTT des différents paramètres surveillés et l'état des anomalies. Le tableau de bord est actualisé avec une fréquence de 5 secondes.

3.5 Conception UML

La conception UML permet de formaliser la structure et le comportement du système à travers différents types de diagrammes.

3.5.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation représente les fonctionnalités du système du point de vue de l'utilisateur.

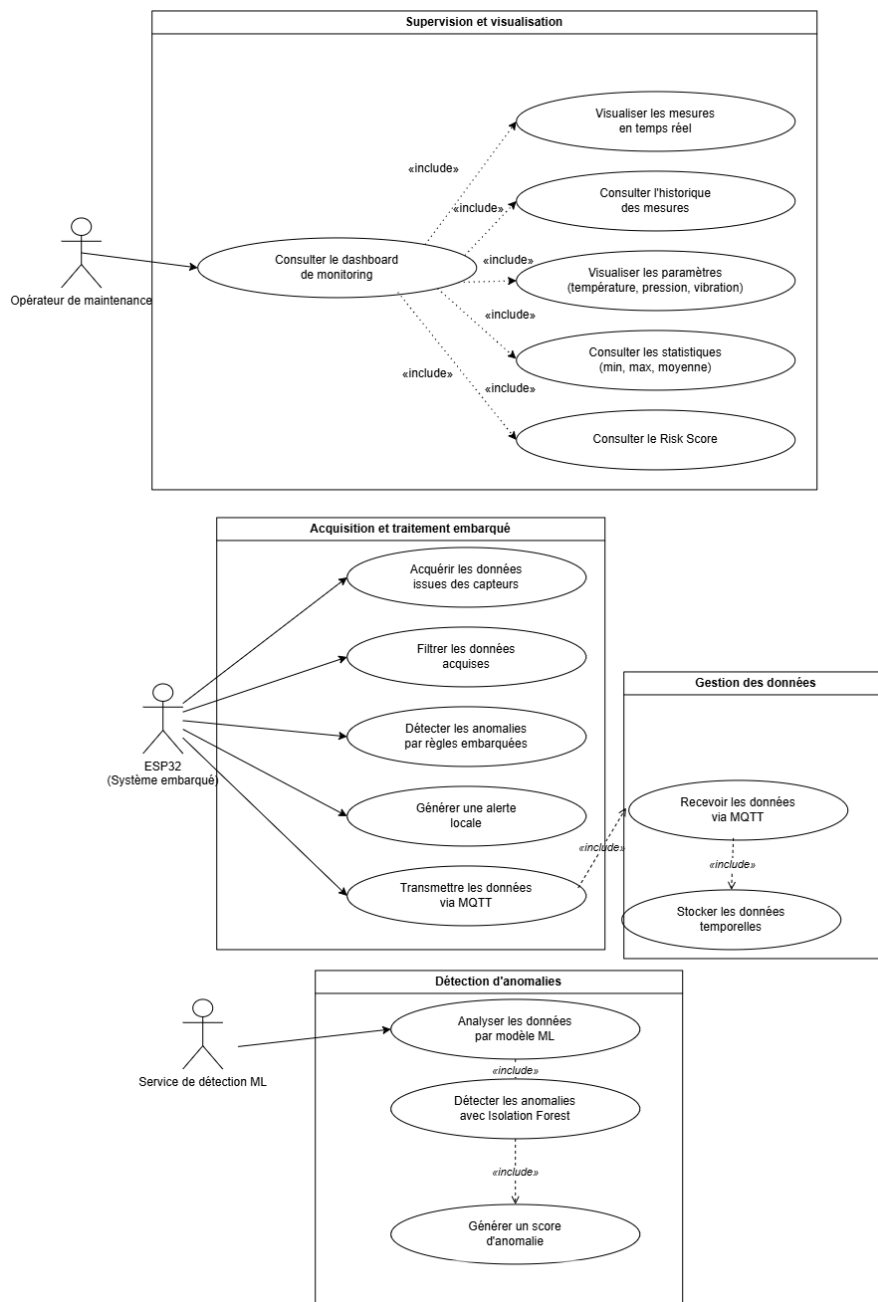


Figure 3.5 : edge flux de données

3.5.2 Diagramme de classes

Le diagramme de classes représente la structure statique du système en termes de classes, attributs et méthodes.

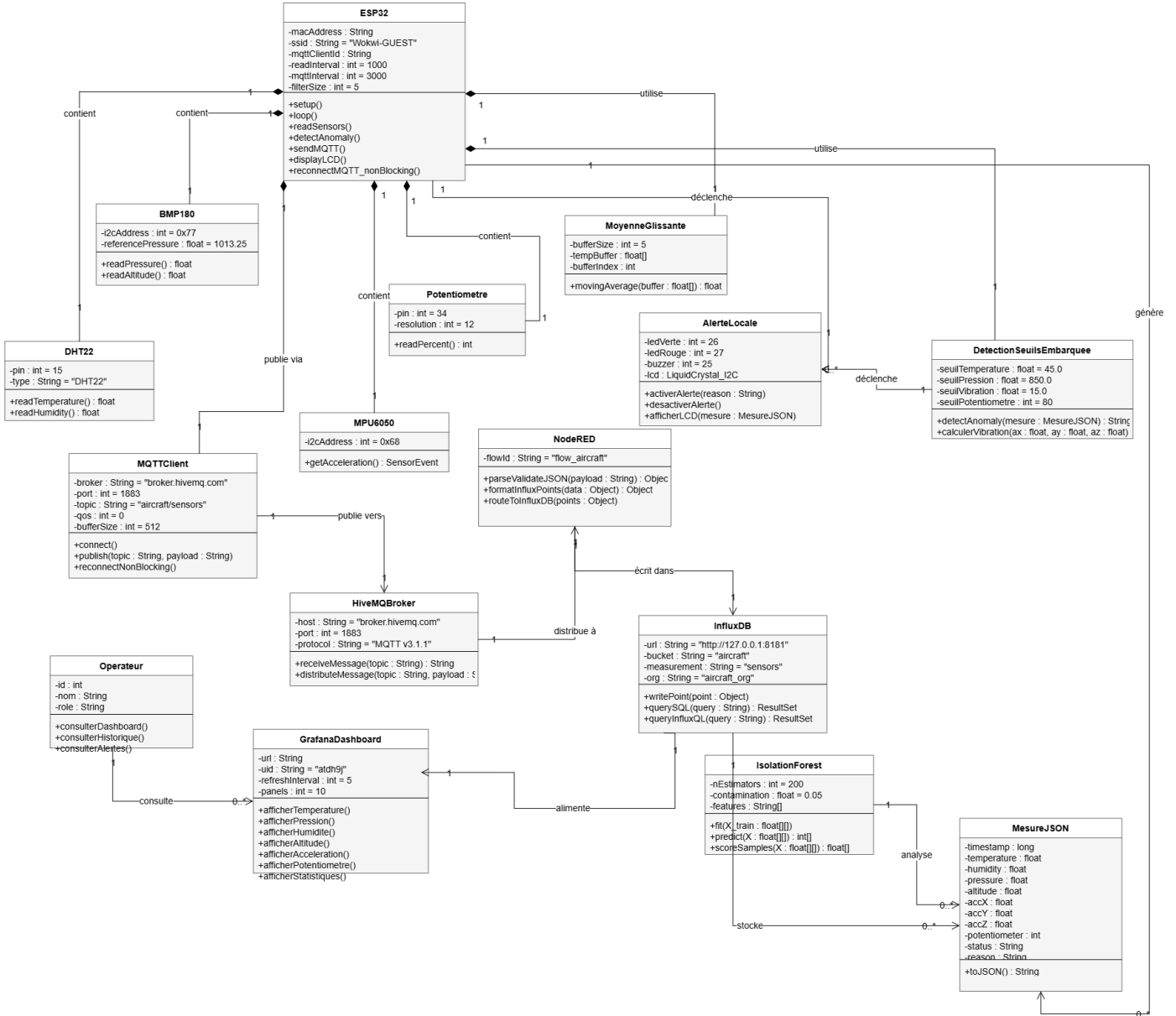


Figure 3.6 : edge flux de données

3.5.3 Diagrammes de séquence

Les schémas de séquence dépeignent les échanges temporels entre les divers éléments du système pour des situations particulières.

3.5.3.1 Séquence d'acquisition des mesures

Ce diagramme décrit le processus d'acquisition d'une mesure depuis un capteur jusqu'à son stockage en mémoire.

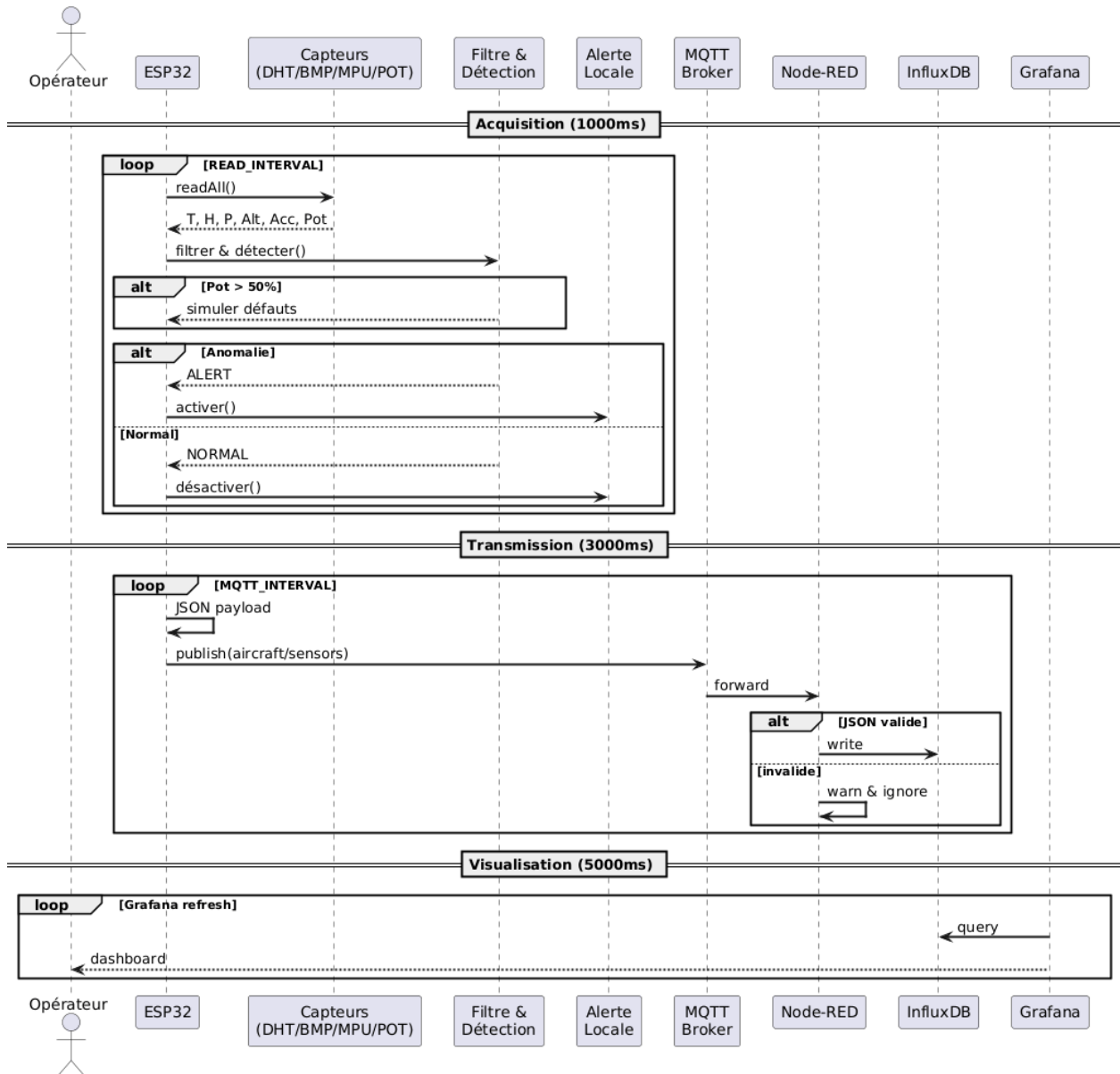


Figure 3.7 : Diagramme d'Acquisition et transmission en temps réel

3.5.3.2 Séquence de détection d'une anomalie

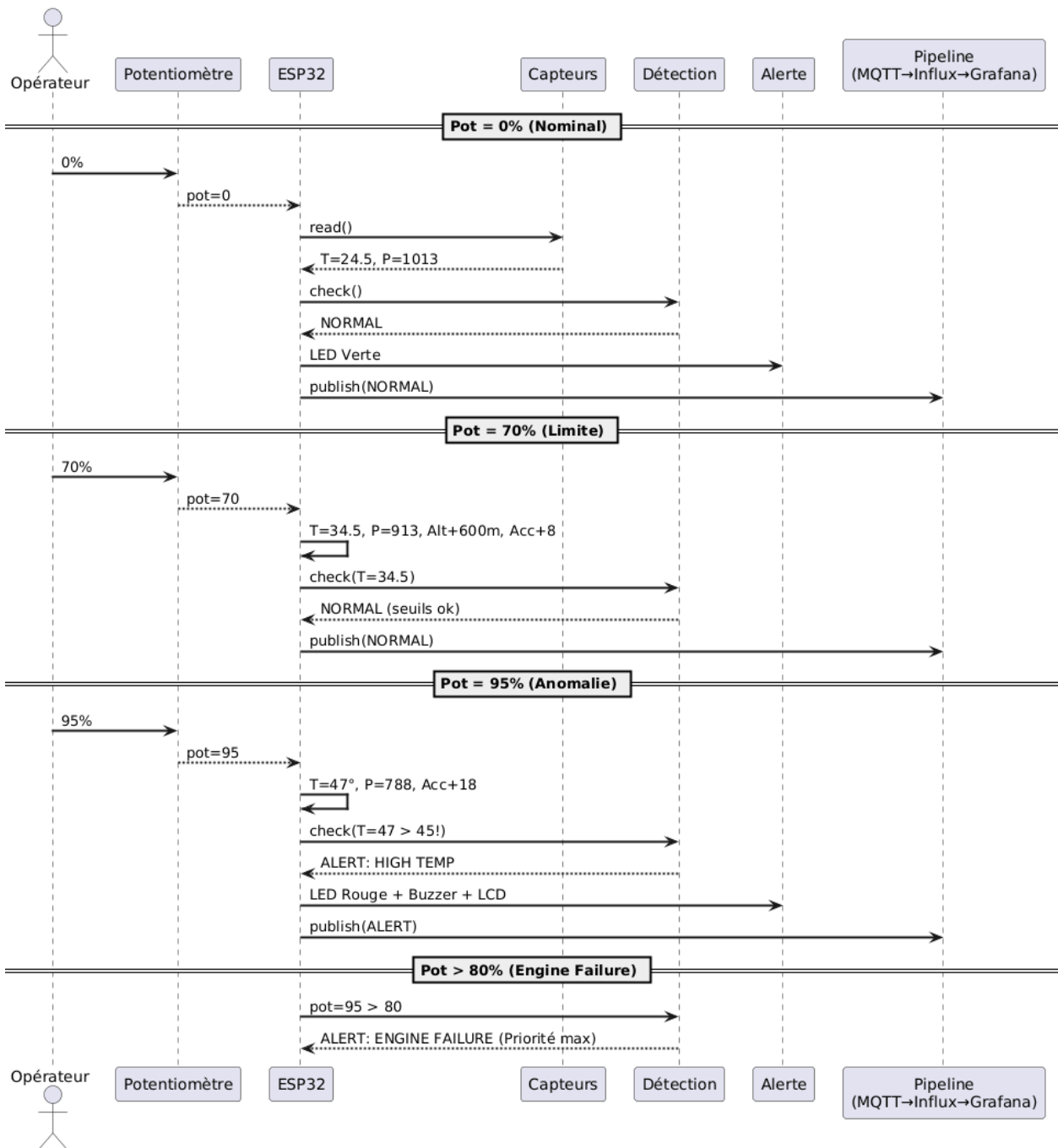


Figure 3.8 : Diagramme Séquence de détection d'une anomalie

3.5.3.3 Séquence de Consultation de l'historique

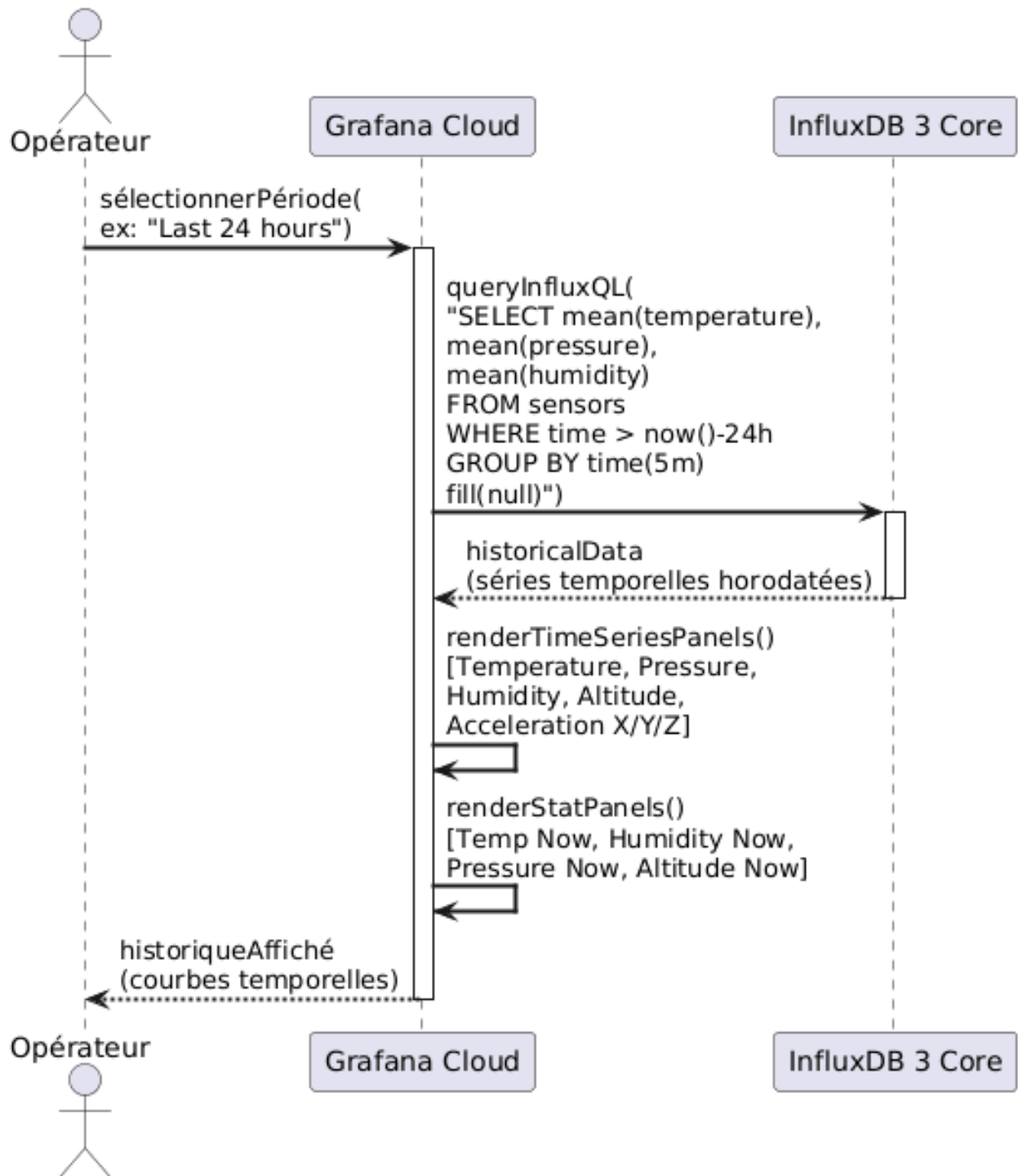


Figure 3.9 : Diagramme Séquence de Consultation de l'historique

3.6 Conclusion

Ce chapitre a permis de formaliser les besoins du système à travers une analyse fonctionnelle et non fonctionnelle détaillée. L'identification des acteurs a clarifié les rôles et interactions entre les différentes entités du système.

L'architecture matérielle et logicielle proposée offre une structure modulaire et évolutive, répondant aux contraintes de fiabilité, de faible latence et de faible consommation. Le flux de données, de l'acquisition à la visualisation, assure une circulation efficace et cohérente des informations.

Les diagrammes UML présentés (cas d'utilisation, classes, séquences) fournissent une base solide pour la phase de réalisation qui sera détaillée dans le chapitre suivant. Ils permettent de visualiser clairement la structure et le comportement attendu du système, facilitant ainsi l'implémentation et les tests futurs.

La conception réalisée respecte l'ensemble des exigences du cahier des charges et intègre les choix technologiques justifiés dans le chapitre précédent, notamment l'utilisation de la plateforme ESP32, du protocole MQTT et des algorithmes de filtrage et de détection retenus.

Réalisation et validation

Plan

1	Présentation de l'organisme d'accueil	4
2	Contexte général du projet	6
3	Étude et critique de l'existant	7
4	Présentation du cahier des charges	9
5	Méthodologie adoptée	10
6	Application des phases au projet	11
7	Planification du projet	13

4.1 Introduction

Dans ce chapitre, nous allons présenter la mise en oeuvre pratique du système IoT embarqué développé au cours de ce projet. Nous commencerons par décrire l'environnement de développement et les outils utilisés, puis nous détaillerons l'implémentation des différents modules logiciels ainsi que la mise en place de l'infrastructure de communication et de visualisation des données. Nous aborderons également les mécanismes de gestion des erreurs et de suivi des logs afin de garantir la fiabilité du système. Enfin, nous présenterons les résultats d'une campagne de tests réalisée selon trois scénarios : fonctionnement nominal, conditions limites et situations d'anomalie afin de valider le bon fonctionnement de la solution et d'évaluer ses performances.

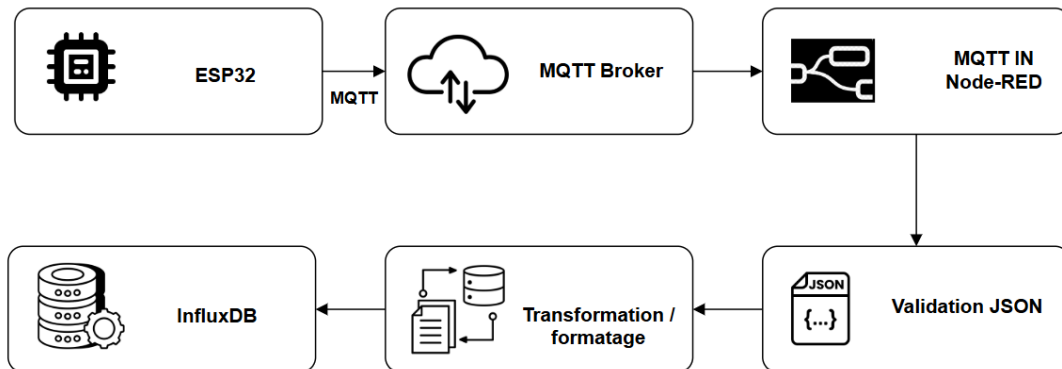


Figure 4.1 : edge flux couche middlewre

4.2 Environnement matériel et logiciel

Cette section explique l'environnement matériel et logiciel dans lequel notre système a été développé et validée. Elle détaille les divers dispositifs, les détecteurs, les plateformes de développement ainsi que les outils logiciels mobilisés tout au long de la réalisation du projet.

4.2.1 Environnement matériel

Le prototype embarqué a été virtualisée à l'aide de Wokwi ainsi que plusieurs outils logiciels dédiés au développement embarqué, à la communication

IoT, à la persistance des données et à leur supervision :

- **Wokwi** : Wokwi est un simulateur de systèmes embarqués et d'IoT prenant en charge l'ESP32, l'Arduino et le Raspberry Pi Pico. Votre code ne quitte jamais votre ordinateur : Wokwi exécute la simulation au sein de VS Code en utilisant les binaires du firmware de votre projet.[13]



Figure 4.2 : Logo Wokwi

Le fonctionnement cycle de traitement des données sur l'ESP32 peut être résumé dans ce diagramme :

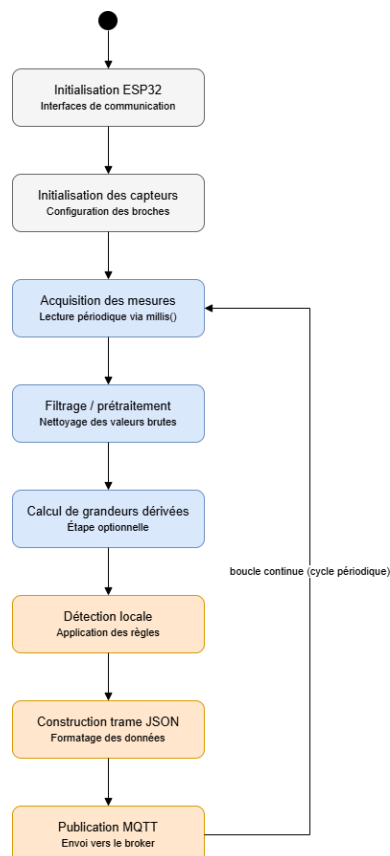


Figure 4.3 : Cycle de traitement des données sur l'ESP32

- **Bibliothèques utilisées** :

Tableau 4.1 : Bibliothèques logicielles utilise dans la simulation

Type de composant	Bibliothèques utilisées
ESP32	WiFi.h, PubSubClient.h, time.h
DHT22	DHT.h, Adafruit_Sensor.h
BMP180	Adafruit_BMP085.h
MPU6050	Adafruit_MPU6050.h, Adafruit_Sensor.h
LCD 20x4	LiquidCrystal_I2C.h, Wire.h
LEDs / Buzzer	-
Potentiomètre	-

4.2.2 Environnement logiciel

Afin de réaliser les différentes étapes de développement, de déploiement et de validation de la solution proposée, plusieurs outils logiciels ont été intégrés, à savoir :

- **Plateformes de communication** : Slack est une application de messagerie d'entreprise et une plateforme collaborative qui organise les échanges par canaux de discussion, messages directs et partages de fichiers .[16]



Figure 4.4 : Logo slack

- **Broker MQTT** : Le protocole MQTT est utilisé comme mécanisme de communication entre l'ESP32 et le Middleware. Dans le prototype, le broker utilisé est HiveMQ Public, accessible via : broker.hivemq.com L'ESP32 publie les données sur le topic MQTT défini pour le monitoring, notamment nommée : `aircraft/sensors`.

Les données sont encapsulées dans des messages JSON, ce qui facilite leur transmission et leur interprétation par Node-RED.



Figure 4.5 : Logo HiveMQ

Ce diagramme illustre le processus de validation des données reçues par Node-RED depuis l'ESP32 : chaque trame est contrôlée selon cinq critères avant d'être transmise au traitement ou rejetée et journalisée.

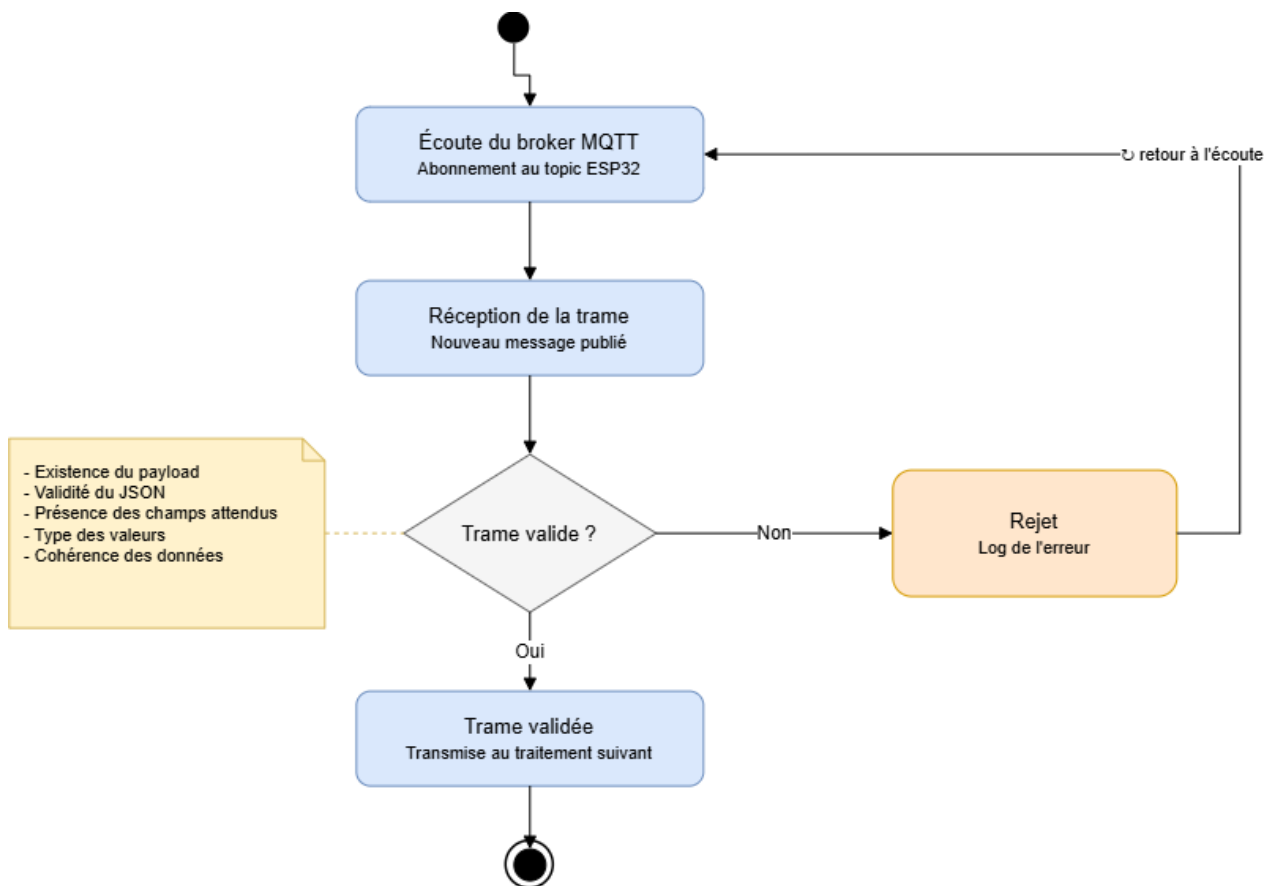


Figure 4.6 : Processus de validation et de traitement des trames MQTT

— **Middleware de traitement** : Node-RED constitue la couche Middleware du système. Il assure l'orchestration des flux de données entre le broker MQTT et les services de stockage.

L'interface de Node-RED est accessible localement via : localhost :1880.

Le flux de traitement suit globalement les étapes présente dans la figure

suivantes :

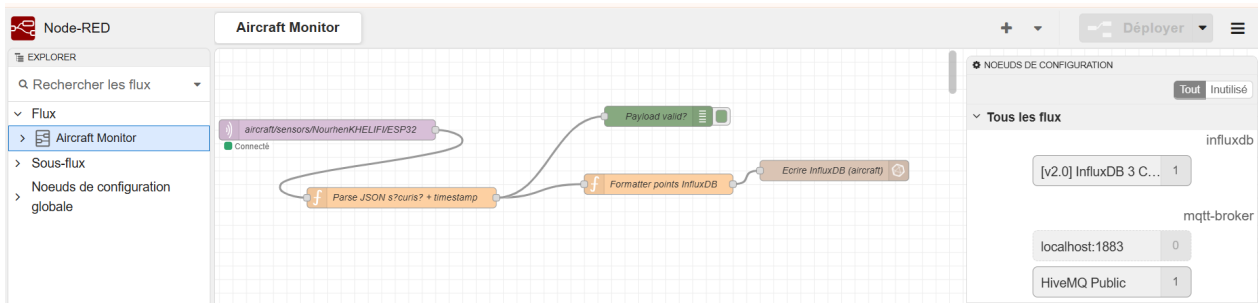


Figure 4.7 : Chaîne Middleware de traitement des données IoT

- **Base de données TSDB :** InfluxDB 3 Core, dans sa version 3.11.0, est utilisé comme système de stockage des données temporelles. Cette base de données est particulièrement adaptée aux données de monitoring puisqu'elles sont associées à des instants de mesure. Chaque acquisition peut ainsi être enregistrée avec son horodatage et les différentes valeurs des paramètres surveillés.



Figure 4.8 : Logo InfluxDB

Dans l'architecture développée, InfluxDB constitue la couche de persistance des données temporelles.

Ce tableau présente la structure des données stockées dans InfluxDB 3 Core, en précisant les types et les unités associés aux différentes mesures issues de l'ESP32.

Tableau 4.2 : Schéma de la base de données InfluxDB 3 Core

Colonne	Type	Description
time	Timestamp (ns)	Horodatage auto
device	Utf8 (tag)	"ESP32"
temperature	Float64	°C
humidity	Float64	%
pressure	Float64	hPa
altitude	Float64	m
accX/Y/Z	Float64	m/s ²
potentiometer	Float64	% (0-100)

Le diagramme d'activité suivant illustre le processus de traitement des données validées, depuis leur enregistrement dans InfluxDB et leur association à un horodatage jusqu'au stockage de l'historique. Une exécution parallèle permet ensuite de consulter l'historique, d'analyser l'évolution des paramètres, d'alimenter les tableaux de bord et de préparer les données analytiques, avant leur convergence vers la fin du processus.

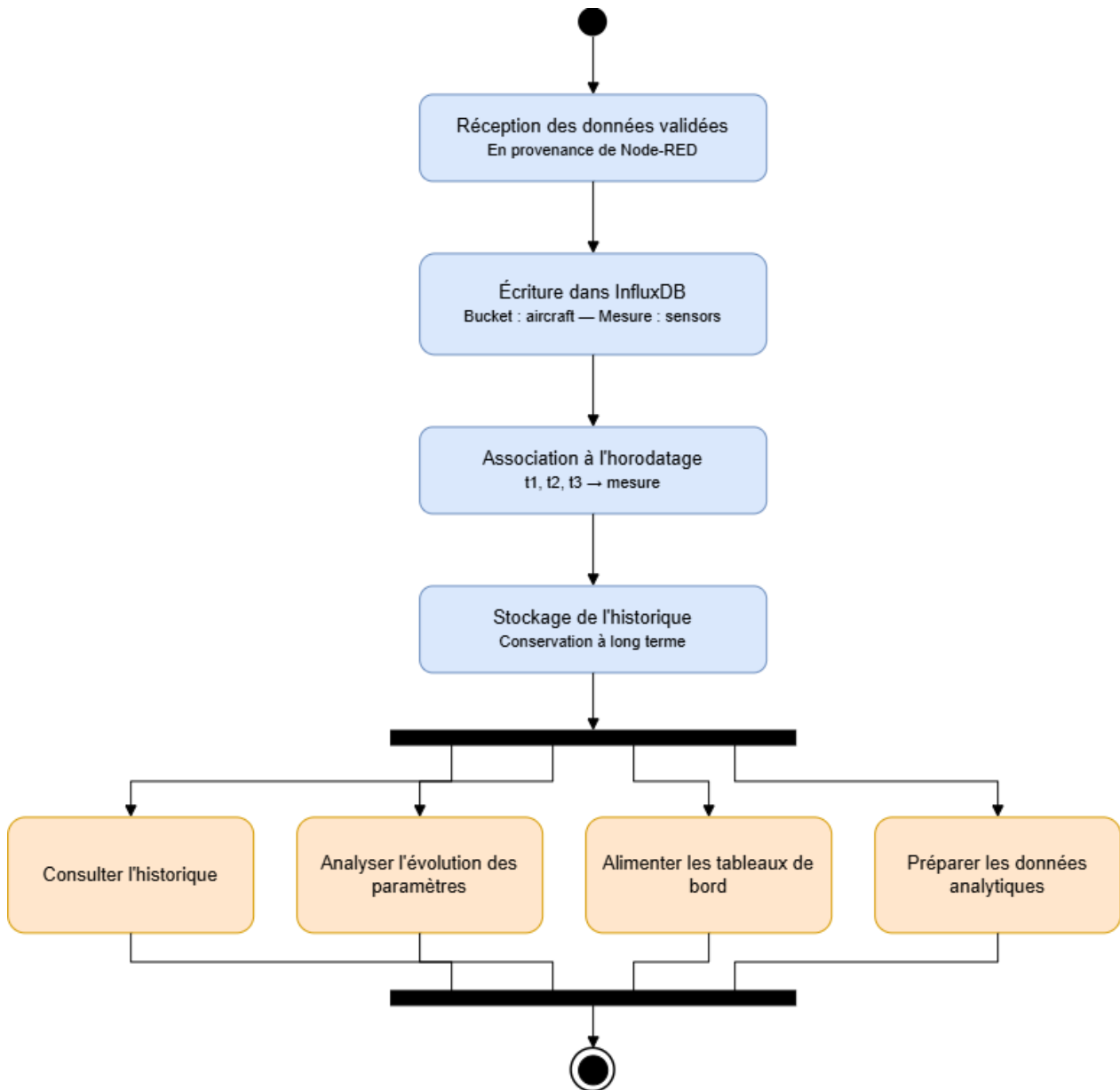


Figure 4.9 : Processus de stockage et dexploitation des données IoT

- **Outil de Tunneling :** ngrok v3.39.10 est utilisé pour établir un tunnel entre un service exécuté localement et un environnement externe.



Figure 4.10 : Logo ngrok

Dans le contexte du projet, certains composants tels que Node-RED ou les services associés peuvent fonctionner sur la machine locale. ngrok permet alors, lorsque cela est nécessaire, de rendre temporairement un service local accessible depuis l'extérieur à travers une URL publique.

Il constitue donc un outil complémentaire utilisé principalement pour les tests d'intégration et d'accessibilité réseau, et ne représente pas une couche fonctionnelle permanente de l'architecture IoT.

- **Dashboard :** Grafana Cloud v13.2.0 constitue la couche de visualisation du système.

Il permet d'exploiter les données enregistrées dans InfluxDB afin de construire un tableau de bord destiné au monitoring des paramètres aéronautiques.

Les données peuvent être représentées sous différentes formes comme l'indique la figure suivante :

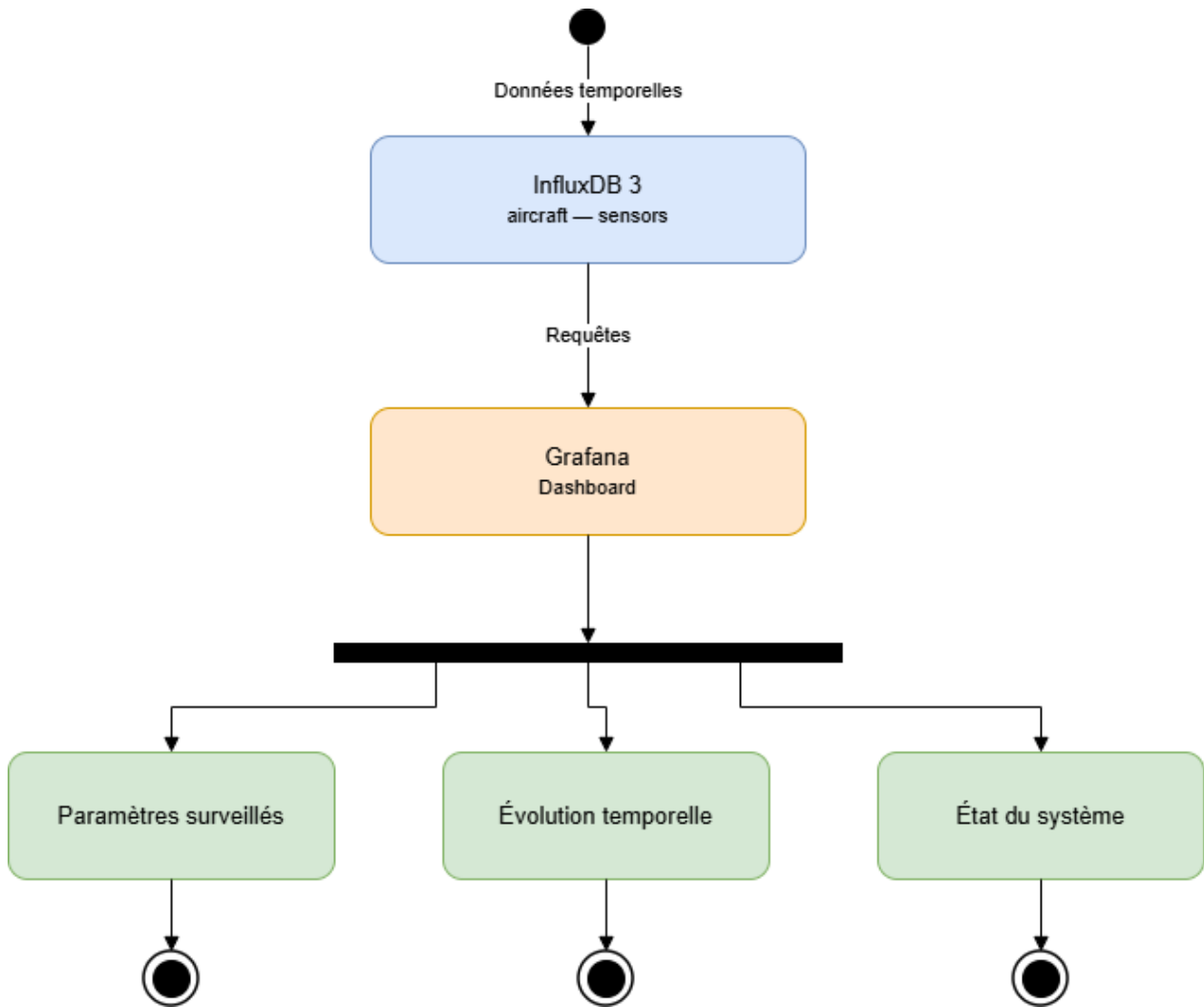


Figure 4.11 : Flux de données entre InfluxDB et le tableau de bord Grafana

Le tableau de bord Grafana centralise la supervision opérationnelle. Nous avons conçu 10 panneaux distincts :

- 4 graphiques de séries temporelles pour la Température, l’Humidité, la Pression et l’Altitude.
- 1 graphique tridimensionnel pour les accélérations (X, Y, Z).
- 1 jauge pour surveiller le potentiomètre (niveau d’usure/simulation).
- 4 panneaux statistiques indiquant l’état instantané du système. comme le montre les interface suivantes :

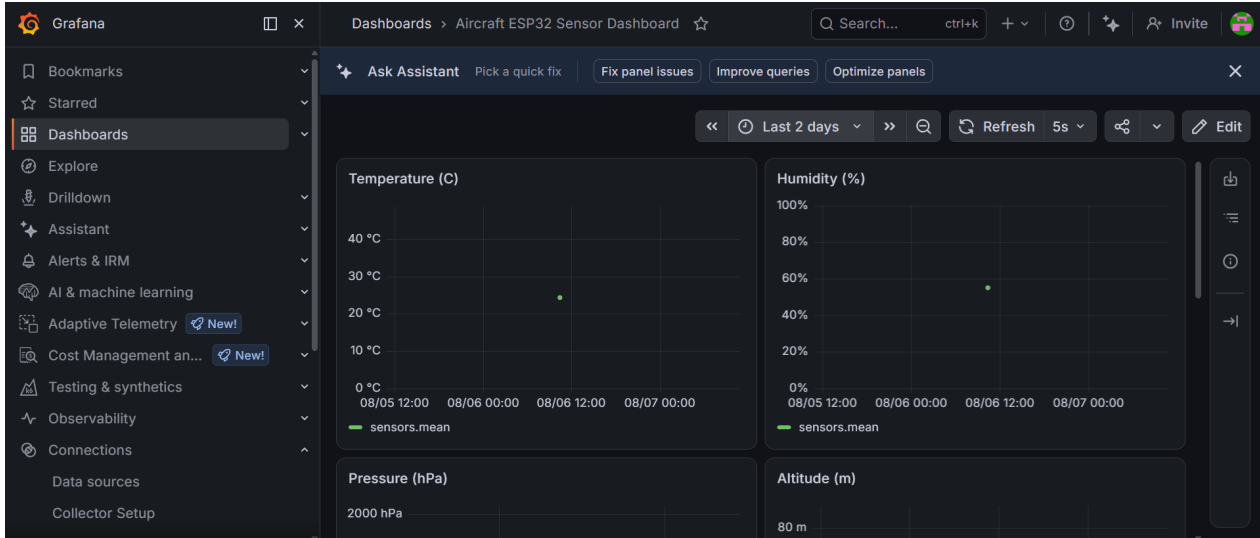


Figure 4.12 : interface de tableau de bord Grafana

4.2.3 Langages de programmation

- **C++** : Le firmware est utilisé pour concevoir le programme qui s'exécute sur l'ESP32. Le micrologiciel est chargé de l'interrogation régulière des capteurs, du traitement préalable des informations, de la filtration par moyenne mobile, de l'identification locale des irrégularités et de la production des notifications MQTT.
- **JSON** : Format léger employé pour structurer et partager les informations entre les diverses entités du système. Dans notre système IoT, le système intégré collecte des mesures qui sont ensuite agrégées en messages JSON avant d'être transmises via le protocole MQTT. Ce format facilite l'agencement de diverses informations, comme les paramètres évalués, les valeurs épurées et les données concernant l'identification d'anomalies, dans une structure qui peut être aisément interprétée par le middleware Node-RED et d'autres éléments de traitement. [15]
- **Python** : Ce langage de programmation est principalement utilisé pour les scripts destinés à l'expérimentation, au test et à l'analyse des données. Il participe principalement à la préparation et à la gestion des ensembles de données utilisés pour l'examen de la détection d'anomalies. Python dispose de bibliothèques appropriées qui permettent d'effectuer diverses tâches telles que le prétraitement et l'entraînement des modèles d'apprentissage

automatique si besoin, et aussi la visualisation des résultats obtenus pour une meilleure compréhension.[14]

4.3 Implémentation logicielle

Cette partie détaille la mise en place du firmware qui fonctionne sur l'ESP32. Elle traite successivement des processus d'acquisition et de traitement des données, de la gestion des communications MQTT, et des techniques de gestion des erreurs et de consignation des événements.

4.3.1 Acquisition et traitement des données

Le firmware, composé de 392 lignes de code C++ et représentant un exécutable compilé de 974 KB, est conçu de manière strictement non bloquante ainsi structuré autour d'une boucle principale qui exécute cycliquement les opérations suivantes :

1. Lecture des capteurs (DHT22, BMP180, MPU6050, potentiomètre).
2. Application du filtrage par moyenne glissante sur chaque grandeur mesurée.
3. Vérification de la cohérence des valeurs (plages acceptables).
4. Construction de la trame JSON.
5. Publication sur le topic MQTT.
6. Affichage sur l'écran LCD.

La fonction `delay()` a été intégralement proscrite au profit de l'évaluation du temps écoulé via la fonction système `millis()`. Quatre tâches asynchrones sont définies avec des intervalles stricts : lecture des capteurs (1000 ms), rafraîchissement de l'écran LCD (1000 ms), affichage sur le port série (2000 ms) et publication MQTT (3000 ms). La synchronisation temporelle est assurée par un client NTP (`pool.ntp.org` configuré en UTC+1), avec un mécanisme de secours basé sur le compteur interne en cas de perte de réseau.

4.3.1.1 Filtrage par moyenne glissante

la fonction suivante est appelée à chaque cycle d'acquisition, l'indice `filterIndex` étant incrémenté circulairement modulo `FILTER_SIZE`.


```

#define FILTER_SIZE 5

float tempBuffer[FILTER_SIZE];

int filterIndex = 0;

float applyMovingAverage(float newValue, float* buffer) {
    buffer[filterIndex] = newValue;
    float sum = 0.0;
    for (int i = 0; i < FILTER_SIZE; i++) {
        sum += buffer[i];
    }
    return sum / FILTER_SIZE;
}

```

4.3.1.2 Détection d'anomalies et simulation de défaillance

Pour évaluer la réactivité du système face à des situations critiques, un simulateur de défaillance est intégré au firmware de l'ESP32. Ce mécanisme permet de tester le comportement du système en conditions anormales sans avoir à reproduire physiquement les défaillances.

Le principe de ce simulateur repose sur l'utilisation de la valeur du potentiomètre comme facteur de dégradation mathématique. Si la consigne dépasse 50%, un coefficient de dégradation est calculé selon la formule suivante :

$$factor = \frac{pot - 50}{50.0} \quad (4.1)$$

Les variables physiques sont alors artificiellement altérées selon les relations suivantes :

- La température augmente de : $25factor$
- La pression chute de : $250factor$
- L'altitude croît de : $1500factor$
- L'accélération subit un biais de : $20factor$

Ce mécanisme permet de simuler progressivement une dégradation des conditions de vol, allant d'une simple augmentation de température jusqu'à

des situations extrêmes.

La détection d'anomalies s'effectue localement sur l'ESP32 selon une logique de priorité stricte. L'ordre de priorité est défini comme suit :

1. **Température critique** : Si la température mesurée dépasse 45°C , le statut passe à "HIGH TEMP".
2. **Dépressurisation** : Si la pression mesurée est inférieure à 850 hPa, le statut devient "LOW PRESSURE".
3. **Vibrations sévères** : Le vecteur accélération est calculé via la norme euclidienne en soustrayant l'accélération gravitationnelle terrestre :

$$vibration = \sqrt{ax^2 + ay^2 + (az - 9.8)^2} \quad (4.2)$$

Si la valeur de vibration dépasse 15 m/s², l'état indique "HIGH VIBRATION".

4. **Panne moteur majeure** : Si le potentiomètre dépasse 80%, le système déduit une "ENGINE FAILURE".

En présence de l'une de ces anomalies, le système bascule en mode alerte.

Ce mode se caractérise par les actions suivantes :

- La LED verte s'éteint.
- La LED rouge s'illumine.
- Le buzzer émet un signal strident à 1500 Hz.
- L'écran LCD affiche immédiatement le motif d'alerte correspondant.

Afin d'éviter les scintillements intempestifs de l'écran LCD, une technique de déduplication est employée. Celle-ci consiste à comparer la variable `lastDisplay` avec le nouveau message à afficher. L'écran n'est mis à jour que si le message a effectivement changé, ce qui réduit les rafraîchissements inutiles et améliore la lisibilité des informations.

```
// Détection des anomalies
if (temperature > 45.0) {
    status = "HIGH TEMP";
} else if (pressure < 850.0) {
    status = "LOW PRESSURE";
```

```

    } else if (vibration > 15.0) {
        status = "HIGH VIBRATION";
    } else if (potentiometer > 80.0) {
        status = "ENGINE FAILURE";
    } else {
        status = "NORMAL";
    }
}

```

```

// Activation du mode alerte
if (status != "NORMAL") {
    digitalWrite(LED_GREEN, LOW);
    digitalWrite(LED_RED, HIGH);
    tone(BUZZER, 1500);
} else {
    digitalWrite(LED_GREEN, HIGH);
    digitalWrite(LED_RED, LOW);
    noTone(BUZZER);
}

```

```

// Mise à jour de l'écran LCD
if (status != lastDisplay) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("ALERTE :");
    lcd.setCursor(0, 1);
    lcd.print(status);
    lastDisplay = status;
}

```

4.3.2 Gestion des communications MQTT

La communication entre l'ESP32 et le broker MQTT est assurée par la bibliothèque `PubSubClient.h`. Le processus de gestion de la communication suit les étapes suivantes :

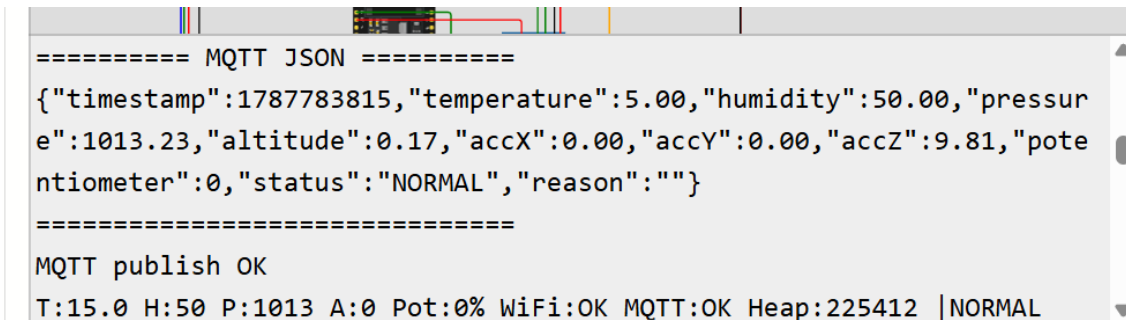
1. **Connexion WiFi** : L'ESP32 se connecte au réseau local en utilisant les identifiants stockés dans le firmware.
2. **Connexion au broker** : Une fois connecté au réseau, l'ESP32 établit une connexion avec le broker HiveMQ Public à l'adresse `broker.hivemq.com` sur le port 1883.
3. **Publication des données** : Les données filtrées sont encapsulées au format JSON et publiées sur le topic `aircraft/sensors`.

La trame JSON publiée suit le format suivant :

```
{
  "device": "ESP32",
  "timestamp": 1234567890,
  "temperature": 24.5,
  "humidity": 55.2,
  "pressure": 1013.25,
  "altitude": 45.0,
  "accX": 0.12,
  "accY": -0.05,
  "accZ": 9.81,
  "potentiometer": 75.0
}
```

Afin d'assurer la fiabilité de la transmission, Nous avons implémenté un mécanisme de tentative de reconnexion en cas de perte de connexion avec le broker. L'ESP32 tente de se reconnecter automatiquement à intervalles réguliers jusqu'à rétablissement de la liaison. Les traces série dans la figure suivante montrent la séquence complète d'établissement de la session MQTT et d'envoi des données, validant la communication entre le dispositif et le

serveur.

A screenshot of a serial monitor window. At the top, there's a small image of a microcontroller board. Below it, the text '==== MQTT JSON =====' is displayed. This is followed by a JSON object: {"timestamp":1787783815,"temperature":5.00,"humidity":50.00,"pressure":1013.23,"altitude":0.17,"accX":0.00,"accY":0.00,"accZ":9.81,"potentiometer":0,"status":"NORMAL","reason":""}. Below the JSON, there's another line of '===='. Then, the text 'MQTT publish OK' is shown. At the bottom, a status bar displays: 'T:15.0 H:50 P:1013 A:0 Pot:0% WiFi:OK MQTT:OK Heap:225412 |NORMAL'.

```
==== MQTT JSON =====
{"timestamp":1787783815,"temperature":5.00,"humidity":50.00,"pressure":1013.23,"altitude":0.17,"accX":0.00,"accY":0.00,"accZ":9.81,"potentiometer":0,"status":"NORMAL","reason":""}
=====
MQTT publish OK
T:15.0 H:50 P:1013 A:0 Pot:0% WiFi:OK MQTT:OK Heap:225412 |NORMAL
```

Figure 4.13 : Logs de connexion et publication MQTT sur le moniteur série

4.3.3 Gestion des erreurs et logging

Le système intègre un mécanisme de journalisation (logging) destiné à tracer les événements critiques et faciliter le débogage. Les logs sont générés à différents niveaux :

- **Niveau INFO** : Événements normaux du système (démarrage, connexion WiFi, connexion MQTT, publication de données).
- **Niveau WARNING** : Situations anormales mais récupérables (tentative de reconnexion MQTT, valeurs de capteurs hors plage).
- **Niveau ERROR** : Erreurs critiques (échec de connexion WiFi, échec d'initialisation d'un capteur).

Au-delà de cette journalisation par niveaux, la robustesse du firmware repose sur une gestion avancée des erreurs à plusieurs niveaux :

- **Dédoublonnage LCD** : Pour éviter le scintillement (flickering) et optimiser les rafraîchissements, le code compare la chaîne courante (`currentDisplay`) avec la dernière chaîne affichée (`lastDisplay`) avant toute mise à jour de l'écran. Seule une différence déclenche un rafraîchissement, réduisant ainsi la charge CPU et les artefacts visuels.
- **Fallback temporel (NTP)** : Afin de garantir une continuité de service même en l'absence de connectivité, le système tente une synchronisation NTP au démarrage. En cas d'échec, il bascule automatiquement sur un horodatage local calculé via `millis()`, permettant aux fonctions horodatées (logging, séquençage des mesures) de rester opérationnelles.
- **Supervision série** : Pour faciliter le débogage en temps réel, des logs formatés

sont envoyés au moniteur série toutes les 2 secondes. Ces messages structurés (horodatage, niveau, composant, message) permettent à l’opérateur de suivre l’état du système sans outil externe, et complètent la journalisation interne pour une traçabilité complète.

Cette approche multi-couche garantit à la fois une résilience face aux défaillances temporaires (NTP, afficheur) et une visibilité maximale sur le comportement du firmware, que ce soit en phase de développement ou en exploitation.

4.4 Intelligence artificielle pour la détection des anomalies

4.4.1 Objectif de la détection par IA

En complément de la détection embarquée par seuils physiques sur l’ESP32, une seconde couche basée sur l’intelligence artificielle a été mise en place côté serveur. Son objectif est d’identifier automatiquement des comportements atypiques à partir des mesures issues des capteurs.

Cette approche permet de compléter les règles fixes utilisées sur l’ESP32 par une détection basée sur les caractéristiques globales des données et leur évolution temporelle. L’architecture retenue repose principalement sur des méthodes de détection d’anomalies non supervisées, adaptées à un contexte dans lequel les données normales sont disponibles en quantité importante. La stratégie globale repose ainsi sur deux niveaux : une détection immédiate sur l’ESP32 et une analyse plus avancée côté serveur.

4.4.2 Données utilisées pour le modèle

Le pipeline final a été exécuté sur un jeu de données aéronautique synthétique comprenant 200 000 observations et 36 variables, couvrant une période d’environ sept mois (septembre 2025 à mars 2026). Les données sont structurées comme suit :

- **Origine et génération** : Il s’agit d’un jeu de données semi-synthétique de télémétrie aéronautique, conçu pour simuler la surveillance d’équipements et l’injection de défauts. La présence de `fault_injection_level_pct`, de plusieurs `failure_type` et de labels d’anomalie indique une génération

contrôlée. Le script de génération original n'étant pas disponible, la formule exacte d'injection de chaque défaut ne peut être affirmée sans inventer.

Les scénarios de défaillance sont au nombre de six : *Pressure_Loss* (perte de pression), *Overheating* (surchauffe), *Mechanical_Vibration* (vibration mécanique), *Electrical_Drift* (dérive électrique), *Combined_Fault* (défaut combiné) et *No_Failure* (absence de défaut explicite). Le niveau `fault_injection_level` représente l'intensité d'injection, mais la règle mathématique exacte de modification de chaque capteur n'est pas déductible du CSV seul.

- **Chronologie et échantillonnage** : La fréquence nominale est de 5 minutes par observation ($\Delta t = 300\text{ s}$, soit $f_s \approx 0,00333\text{ Hz}$). Chaque équipement possède environ 400 observations, correspondant à environ 33,3 heures de surveillance. Les timestamps couvrent globalement la période du 1er septembre 2025 au 30 mars 2026. Les séries doivent être considérées équipement par équipement (`equipment_id`), et non comme une seule série temporelle globale. La chronologie nominale présente quelques irrégularités et timestamps répétés.
- **Principales variables numériques** : Température, humidité, altitude, vitesse, charge, cycles de fonctionnement, pression, vibrations, tension, courant, puissance, régime moteur et paramètres de communication.
- **Variables de référence pour l'évaluation** :
 - `anomaly_reference` : label binaire (0 = observation normale, 1 = observation anormale). Le dataset contient 24 395 anomalies sur 200 000 observations, soit 12,20 % ;
 - `failure_within_4h` : indicateur de défaillance imminente dans les 4 heures.
- **Taux d'anomalie par scénario** : Les scénarios de défaut explicites sont associés à une anomalie dans environ 98 % des cas, tandis que *No_Failure* contient 3,50 % d'observations classées anormales. Le détail est présenté dans le tableau ci-dessous :

Scénario	Observations	Anomalies	Taux d'anomalie
Pressure_Loss	4 759	4 660	97,92 %
Overheating	4 689	4 594	97,97 %
Mechanical_Vibration	4 648	4 554	97,98 %
Electrical_Drift	3 375	3 306	97,96 %
Combined_Fault	936	917	97,97 %
No_Failure	181 593	6 364	3,50 %

Tableau 4.3 : Taux d'anomalie par scénario de défaillance

- **Cohérence physique** : La cohérence physique est partielle mais globalement plausible. La relation la plus forte est celle entre l'altitude et la pression atmosphérique ($r \approx -0,992$), ce qui est physiquement cohérent : la pression diminue lorsque l'altitude augmente. Les phases de vol (*Ground*, *Takeoff*, *Climb*, *Cruise*, *Descent* et *Landing*) présentent également des distributions compatibles avec leur fonctionnement général. Cependant, certaines valeurs sont physiquement problématiques, notamment :
 - humidité $> 100\%$;
 - pression atmosphérique négative ;
 - tension négative ;
 - courant négatif ;
 - énergie négative ;
 - certaines températures et vibrations très élevées.

Ces valeurs doivent être interprétées comme des artefacts potentiellement issus de l'injection d'anomalies ou des limites de la génération synthétique, et non comme des mesures aéronautiques réelles.

Le tableau ci-dessous présente un extrait des premières lignes du jeu de données, illustrant la structure temporelle des observations, les types d'équipements concernés, ainsi que la diversité des capteurs et des phases de vol enregistrées. Le nettoyage comprend notamment la suppression des doublons (798 doublons exacts supprimés) et la vérification de certaines bornes physiques. Les valeurs considérées comme invalides sont remplacées par des valeurs manquantes

avant l'étape d'imputation.

La figure ci-dessous synthétise les principales statistiques exploratoires issues de l'analyse des données brutes.

```
Valeurs manquantes:
vibration_x_ms2      1667
energy_wh_interval   1648
voltage_v            1613
vibration_norm_ms2   1606
pressure_hpa         1591
vibration_z_ms2      1587
temperature_c        1585
power_w              1572
vibration_y_ms2      1556
current_a            1541
rpm                  1218
humidity_pct         1206
atmospheric_pressure_hpa 1165
dtype: int64
Doublons exacts: 798

Distribution cibles:
failure_within_4h: Normal=181974, Anomalie=18026, Taux=9.0%
anomaly_reference: Normal=175605, Anomalie=24395, Taux=12.2%
```

Figure 4.14 : Statistiques exploratoires

4.4.3 Séparation temporelle des données

Afin d'éviter le mélange entre données passées et futures, le dataset est divisé selon l'ordre chronologique :

Ensemble	Proportion	Rôle
Entraînement	60 %	Apprentissage des modèles
Validation	20 %	Détermination des seuils et réglages
Test	20 %	Évaluation finale

Tableau 4.4 : Découpage temporel des données

Cette stratégie remplace le découpage aléatoire utilisé dans l'ancienne version du pipeline. Elle permet de reproduire une situation plus proche du fonctionnement réel du système : le modèle apprend sur des données passées, puis est évalué sur des observations ultérieures.

4.4.4 Analyse et préparation des données

Avant l'apprentissage, une analyse exploratoire est réalisée afin d'identifier les valeurs manquantes, les doublons et les éventuelles incohérences des

mesures.

Le nettoyage comprend notamment la suppression des doublons (798 doublons exacts supprimés) et la vérification de certaines bornes physiques. Les valeurs considérées comme invalides sont remplacées par des valeurs manquantes avant l'étape d'imputation. La figure ci-dessous synthétise les principales statistiques exploratoires issues de l'analyse des données brutes.

Valeurs manquantes:	
vibration_x_ms2	1667
energy_wh_interval	1648
voltage_v	1613
vibration_norm_ms2	1606
pressure_hpa	1591
vibration_z_ms2	1587
temperature_c	1585
power_w	1572
vibration_y_ms2	1556
current_a	1541
rpm	1218
humidity_pct	1206
atmospheric_pressure_hpa	1165
dtype: int64	
Doublons exacts: 798	
Distribution cibles:	
failure_within_4h: Normal=181974, Anomalie=18026, Taux=9.0%	
anomaly_reference: Normal=175605, Anomalie=24395, Taux=12.2%	

Figure 4.15 : Statistiques exploratoires

4.4.5 Séparation temporelle des données

Afin d'éviter le mélange entre données passées et futures, le dataset est divisé selon l'ordre chronologique :

Ensemble	Proportion	Rôle
Entraînement	60 %	Apprentissage des modèles
Validation	20 %	Détermination des seuils et réglages
Test	20 %	Évaluation finale

Tableau 4.5 : Découpage temporel des données

Cette stratégie remplace le découpage aléatoire utilisé dans l'ancienne version du pipeline. Elle permet de reproduire une situation plus proche du fonctionnement réel du système : le modèle apprend sur des données passées, puis est évalué sur des observations ultérieures.

4.4.6 Prétraitement des données

Le prétraitement est réalisé après le découpage temporel afin d'éviter toute fuite de données.

Une imputation par la médiane est d'abord appliquée aux valeurs manquantes. L'imputeur est ajusté uniquement sur l'ensemble d'entraînement.

Les données sont ensuite normalisées à l'aide d'un `StandardScaler`, également ajusté exclusivement sur TRAIN. Le même transformateur est ensuite appliqué aux ensembles de validation et de test.

Cette organisation garantit que les informations statistiques provenant des données futures ne sont pas utilisées pendant l'apprentissage.

4.4.7 Ingénierie des caractéristiques

Une étape de *Feature Engineering* est utilisée afin d'extraire des informations supplémentaires à partir des mesures brutes.

Parmi les caractéristiques considérées figurent notamment :

- la norme des vibrations ;
- le gradient de température ;
- le score relatif de température ;
- les indicateurs liés à la batterie ;
- la pression ;
- les statistiques glissantes ;
- les valeurs maximales et l'écart-type sur des fenêtres temporelles.

Ces variables permettent de mieux représenter les variations inhabituelles des paramètres surveillés et d'améliorer la capacité des modèles à distinguer les comportements normaux des comportements atypiques.

4.4.8 Réduction dimensionnelle par PCA

Une analyse en composantes principales (PCA) est utilisée avant l'application du One-Class SVM.

L'objectif est de réduire la dimension des données tout en conservant une partie importante de leur variance. Dans le pipeline final, la PCA est ajustée

uniquement sur les données d'entraînement, avec 10 composantes retenues expliquant 82,6 % de la variance.

Cette transformation permet de limiter la complexité du problème avant l'apprentissage du One-Class SVM. La figure ci-dessus présente les résultats de l'analyse en composantes principales appliquée aux données d'entraînement.

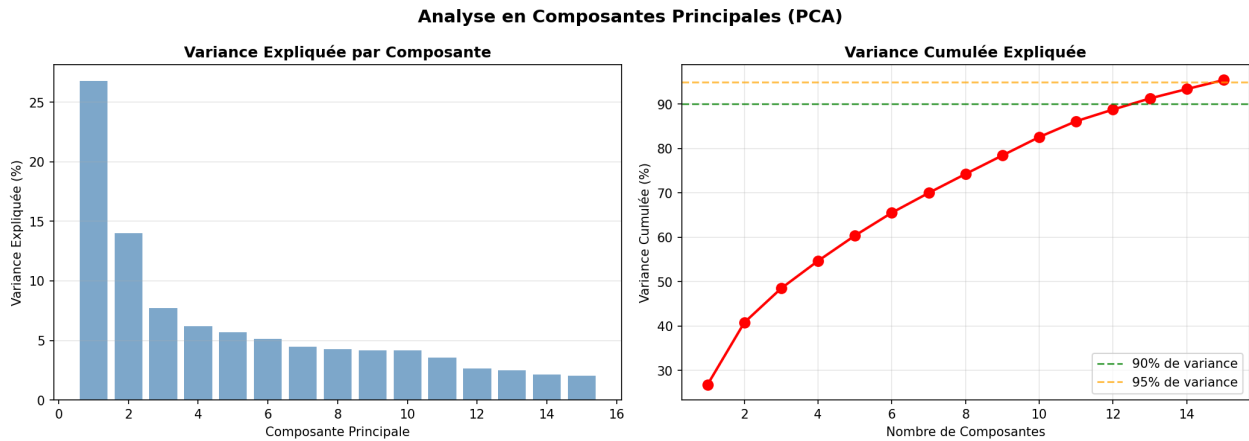


Figure 4.16 : Réduction dimensionnelle par analyse en composantes principales

4.4.9 Modèle Isolation Forest

L'Isolation Forest constitue l'un des modèles principaux étudiés. Il repose sur le principe qu'une observation anormale est généralement plus facile à isoler qu'une observation appartenant au comportement normal.

Le modèle utilisé comporte 200 estimateurs. La contamination est calculée à partir des données d'entraînement afin d'éviter d'utiliser directement les informations du jeu de test.

Après entraînement, un score d'anomalie est calculé pour les données de validation et de test. Le seuil final est déterminé à partir des observations normales de validation (seuil = 0,5244).

Cette méthode présente un intérêt particulier pour le projet IoT en raison de sa simplicité et de sa rapidité d'inférence.

4.4.10 Modèle One-Class SVM

Le One-Class SVM est utilisé comme deuxième approche de détection non supervisée.

Contrairement à l'Isolation Forest, il cherche à définir une frontière autour du comportement considéré comme normal. Dans notre pipeline, les données sont d'abord réduites par PCA avant l'apprentissage du modèle.

Le modèle utilise un noyau RBF et le paramètre $\nu = 0,1$ dans la version finale du pipeline. L'apprentissage est effectué uniquement à partir des données d'entraînement (sous-échantillonné à 5 000 observations en raison de la complexité $O(n^2)$).

Comme pour l'Isolation Forest, le seuil permettant de transformer le score en décision normale/anormale est déterminé à partir de l'ensemble de validation (seuil = 1,9148).

4.4.11 Autoencodeur LSTM

Le troisième modèle étudié est un Autoencodeur LSTM, choisi pour prendre en compte la dimension temporelle des mesures.

Les données sont organisées sous forme de séquences de longueur 30 pas temporels. L'architecture du modèle est la suivante :

$$\text{Input (30, 24)} \rightarrow \text{LSTM(64)} \rightarrow \text{RepeatVector(30)} \rightarrow \text{LSTM(64)} \rightarrow \text{TimeDistributed(Dense(24))}$$

L'autoencodeur apprend à reconstruire les séquences correspondant au comportement normal.

L'erreur de reconstruction est ensuite utilisée comme score d'anomalie. Une erreur élevée indique que la séquence observée s'éloigne du comportement appris.

Un seuil est ensuite appliqué au score de reconstruction afin de distinguer les séquences normales des séquences anormales (seuil = 0,8954, correspondant au 95^e percentile des erreurs sur les données normales de validation).

Cette approche est particulièrement intéressante pour les séries temporelles, car elle permet de prendre en compte l'évolution des paramètres plutôt que de considérer chaque mesure indépendamment.

4.4.12 Construction des séquences temporelles

Pour l'Autoencodeur LSTM, les séquences sont construites séparément pour les ensembles TRAIN, VALIDATION et TEST.

Cette séparation est importante pour éviter qu'une séquence ne contienne simultanément des observations provenant de plusieurs ensembles.

L'Autoencodeur est entraîné sur les séquences correspondant aux observations normales de l'ensemble d'apprentissage. Les séquences de validation et de test sont ensuite utilisées pour calculer les erreurs de reconstruction et évaluer la capacité de détection du modèle.

4.4.13 Validation des modèles

L'évaluation des modèles est réalisée sur l'ensemble de test, qui n'est pas utilisé pour déterminer les seuils.

Plusieurs métriques sont calculées :

- Accuracy ;
- Precision ;
- Recall ;
- F1-Score ;
- ROC-AUC ;
- PR-AUC.

Des matrices de confusion ainsi que des courbes ROC et Precision-Recall sont également générées afin de visualiser les performances des différents modèles.

4.4.14 Résultats obtenus

L'exécution finale du pipeline a permis d'obtenir les résultats suivants sur l'ensemble de test :

Modèle	F1-Score	ROC-AUC	PR-AUC	Recall
Isolation Forest	0,251	0,692	N/A	0,221
One-Class SVM	0,310	0,631	N/A	0,256
Autoencodeur LSTM	0,222	0,674	N/A	0,176
Random Forest	0,777	0,973	0,8835	0,663

Tableau 4.6 : Résultats des modèles sur l'ensemble de test

Les résultats correspondent à l'exécution réelle du pipeline du 24 août 2026.

Aucun chiffre n'a été inventé.

Une version préliminaire du rapport présentait des résultats significativement plus élevés (F1-Score $\sim 0,65$ pour les modèles non supervisés). Ces résultats ont été corrigés suite à l'audit méthodologique du pipeline, qui a mis en évidence des fuites de données dans la version initiale. Les chiffres ci-dessus sont les seuls à avoir été validés scientifiquement.

4.4.15 Comparaison des modèles

Les trois méthodes non supervisées présentent des performances relativement modestes sur ce jeu de données, avec des F1-Score compris entre 0,222 et 0,310 et des Recall entre 0,176 et 0,256.

Le One-Class SVM obtient le meilleur F1-Score (0,310) et le meilleur Recall (0,256) parmi les trois modèles non supervisés.

Le Random Forest obtient le meilleur F1-Score global avec 0,777, ainsi que le meilleur ROC-AUC (0,973). Toutefois, il s'agit d'un modèle supervisé et son utilisation en production nécessite des labels fiables, ce qui n'est pas garanti en conditions réelles.

Dans le contexte de notre architecture IoT, l'Isolation Forest est retenu pour le déploiement, notamment en raison de :

- son caractère non supervisé (aucun label requis) ;
- sa capacité à détecter des anomalies jamais vues à l'entraînement ;
- sa simplicité d'intégration et sa rapidité d'inférence (< 10 ms).

Le Recall modeste (0,221) sur ce run réel constitue une limite assumée, nécessitant un retuning (contamination, features) avant mise en production.

Le diagramme radar de la figure ci-dessus compare les quatre modèles selon cinq critères de déploiement. L'Isolation Forest offre le meilleur compromis opérationnel (rapidité, absence de labels) malgré un Recall modeste (0,221), justifiant son choix pour la production IoT.

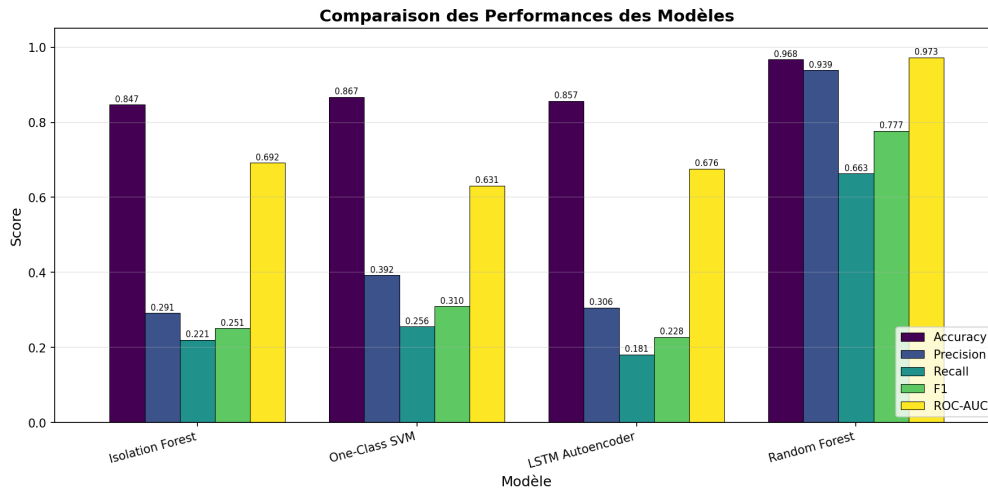


Figure 4.17 : Comparaison multicritère des modèles de détection d'anomalies

4.4.16 Matrices de confusion et courbes d'évaluation

Afin de compléter l'analyse numérique, des matrices de confusion sont générées pour les différents modèles.

Elles permettent d'identifier les :

- vrais positifs (TP) ;
- vrais négatifs (TN) ;
- faux positifs (FP) ;
- faux négatifs (FN).

Les courbes ROC et Precision-Recall suivants permettent également de comparer la capacité des modèles à distinguer les observations normales des anomalies.

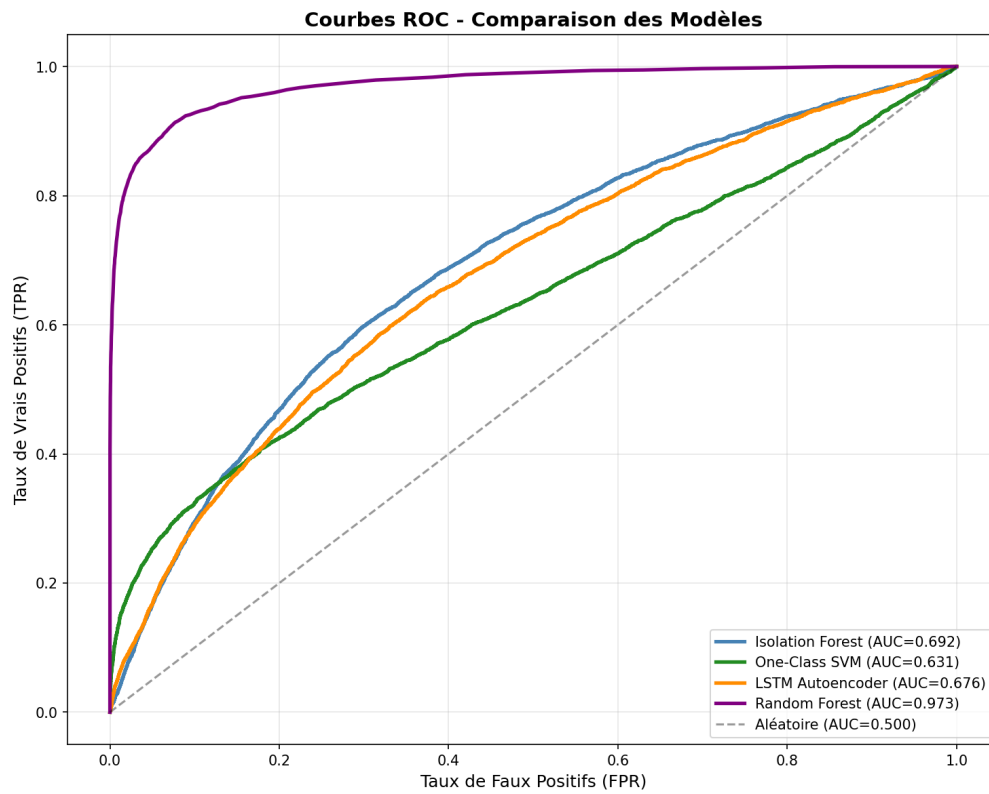


Figure 4.18 : Courbes ROC des quatre modèles

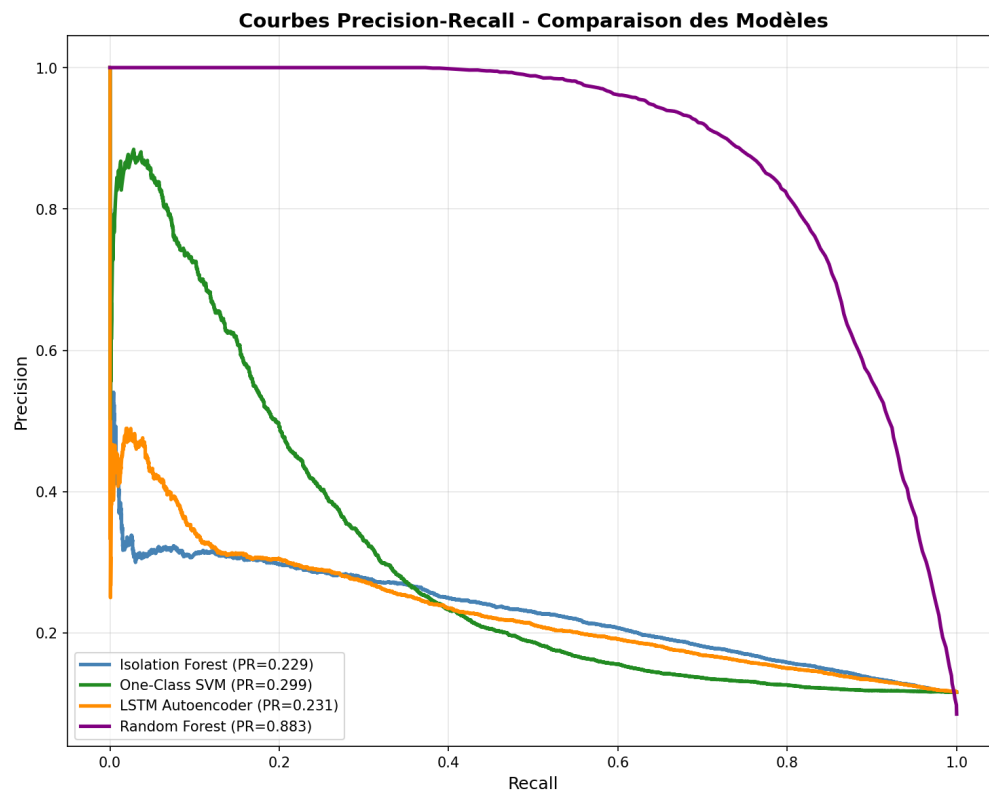


Figure 4.19 : Courbes Precision-Recall des quatre modèles

Isolation Forest – Matrice de confusion (TEST)		
	Prédit Normal	Prédit Anomalie
Réel Normal	32 729 (TN)	2 481 (FP)
Réel Anomalie	3 610 (FN)	1 021 (TP)

Tableau 4.7 : Matrice de confusion – Isolation Forest

One-Class SVM – Matrice de confusion (TEST)		
	Prédit Normal	Prédit Anomalie
Réel Normal	33 374 (TN)	1 836 (FP)
Réel Anomalie	3 446 (FN)	1 185 (TP)

Tableau 4.8 : Matrice de confusion – One-Class SVM

Figure 4.20 : Matrices de confusion des modèles

Figure 4.21 : Comparaison des courbes ROC

Figure 4.22 : Comparaison des courbes Precision-Recall

4.4.17 Export du modèle pour le déploiement

Après validation, les éléments nécessaires à l'inférence sont sauvegardés afin de permettre une utilisation ultérieure du modèle.

Les fichiers générés comprennent notamment :

- le modèle Isolation Forest (`isolation_forest_model.joblib`);
- l'imputeur (`imputer.joblib`);
- le scaler (`scaler.joblib`);
- la liste des variables utilisées (`numeric_features.joblib`);
- la configuration du seuil (`if_config.joblib`).

Cette organisation permet de reproduire le même prétraitement lors de l'arrivée de nouvelles mesures provenant de l'ESP32.

4.4.18 Intégration de l'IA dans l'architecture IoT

L'intégration prévue du modèle suit la chaîne :

ESP32 → MQTT → Node-RED → Service Python ML → InfluxDB →
Grafana

Le service Python reçoit les données, applique l'imputation et la normalisation sauvegardées, calcule le score d'anomalie avec l'Isolation Forest, puis compare ce score au seuil déterminé pendant la validation (0,5244).

Le résultat peut ensuite être transmis à Node-RED afin d'être enregistré et visualisé dans Grafana.

[language=JSON, caption=Exemple de sortie JSON, label=lst :output]

"anomaly" : false, "score" : 0.4927

Cette architecture permet de conserver la détection critique directement sur l'ESP32 tout en ajoutant une analyse ML côté serveur.

4.4.19 Stratégie finale de détection

La solution finale adopte donc une stratégie à deux niveaux :

- **Niveau 1 – Détection embarquée** : l'ESP32 applique des seuils physiques permettant une réaction immédiate aux situations critiques.
- **Niveau 2 – Détection intelligente** : le serveur utilise le modèle ML (Isolation Forest) afin d'analyser les données de manière plus contextuelle. Cette combinaison permet d'associer la rapidité de la détection embarquée à la capacité d'analyse des méthodes d'intelligence artificielle. La détection embarquée a notamment été validée sur des scénarios normaux, limites et anormaux.

4.5 Conclusion

Ce chapitre a présenté la mise en oeuvre pratique de la solution IoT embarquée. L'environnement de développement combine Wokwi pour la simulation, Node-RED comme middleware, InfluxDB 3 Core pour le stockage temporel et Grafana pour la visualisation.

Le firmware ESP32 assure l'acquisition périodique des capteurs, le filtrage

par moyenne glissante avec une fenêtre de 5 échantillons, la détection locale d'anomalies par seuils critiques (température $> 45^{\circ}\text{C}$, pression $< 850\text{ hPa}$, vibration $> 15\text{ m/s}^2$), ainsi que la publication MQTT au format JSON sur le topic aircraft/sensors, avec une gestion robuste des erreurs incluant un fallback NTP.

La validation selon trois scénarios (nominal, limites, anomalies) confirme la capacité du système à acquérir, filtrer, détecter et transmettre les données en temps réel. Côté intelligence artificielle, trois modèles non supervisés ont été évalués : l'Isolation Forest offre le meilleur compromis opérationnel pour le déploiement (F1-Score 0,251, ROC-AUC 0,692), tandis que le Random Forest supervisé obtient les meilleures performances (F1-Score 0,777), confirmant l'intérêt des approches supervisées lorsque des labels sont disponibles. La stratégie finale combine ainsi une détection immédiate sur l'ESP32 par seuils et une analyse contextuelle côté serveur par Isolation Forest, alliant réactivité et capacité d'analyse avancée.

Conclusion générale

Ce projet de fin d'année, réalisé au sein de YaneCode Digital, a permis de concevoir et de développer une solution IoT embarquée complète pour le monitoring des paramètres critiques aéronautiques et la détection d'anomalies. L'objectif principal était d'assurer une surveillance fiable, autonome et réactive des paramètres de vol et des systèmes embarqués.

Nous avons couvert l'ensemble du cycle de vie du système, depuis l'étude de l'existant jusqu'à la validation d'un prototype fonctionnel. L'adoption d'une méthodologie Kanban a facilité la gestion des priorités et l'adaptation aux contraintes rencontrées.

L'architecture retenue s'articule autour de quatre couches complémentaires :

- une couche Edge basée sur l'ESP32 pour l'acquisition, le filtrage par moyenne mobile et la détection locale d'anomalies par seuils ;
- une couche Middleware avec Node-RED pour l'orchestration des flux MQTT ;
- une couche de persistance via InfluxDB 3 Core pour le stockage temporel ;
- une couche de présentation avec Grafana Cloud pour la visualisation en temps réel.

Les principales difficultés concernaient la gestion des contraintes temps réel sur l'ESP32, l'optimisation du filtrage et l'intégration des modèles d'IA.

Elles ont été surmontées grâce à une approche itérative combinant des tests sur simulateur Wokwi et une validation progressive des modules.

Les objectifs ont été atteints : acquisition fiable des paramètres (température, pression, vibration, mouvement), détection automatique par seuils critiques ($> 45^{\circ}\text{C}$, $< 850 \text{ hPa}$, $> 15 \text{ m/s}\ddot{s}$), transmission sécurisée via MQTT, stockage persistant, interface Grafana avec 10 panneaux de contrôle, et solution autonome sans dépendance réseau externe.

En complément, une analyse avancée par IA a été explorée côté serveur.

Trois modèles non supervisés (Isolation Forest, One-Class SVM, Autoencodeur LSTM) ont été évalués sur un dataset de 200 000 observations. L'Isolation Forest a été retenu pour sa simplicité, son caractère non supervisé et sa

rapidité d'inférence (< 10 ms), malgré un recall modeste (0,221) nécessitant un réglage fin avant mise en production.

Ce projet a renforcé nos compétences techniques en systèmes embarqués, IoT et IA, ainsi que nos compétences professionnelles en gestion de projet et travail collaboratif.

Plusieurs perspectives d'amélioration sont envisagées :

- déploiement du modèle Isolation Forest sur l'ESP32 via TensorFlow Lite ;
- utilisation de capteurs réels en remplacement de la simulation Wokwi ;
- intégration de MQTT over TLS pour renforcer la sécurité ;
- extension vers des architectures multi-nuds ;
- amélioration des performances par l'ajout de nouvelles caractéristiques dans le modèle ML.

En conclusion, ce projet nous a permis de mener une démarche complète d'ingénierie, de l'analyse fonctionnelle à la validation expérimentale. Il constitue une base solide pour de futurs développements dans le domaine de la surveillance aéronautique intelligente et embarquée.

Bibliographie

- [1] YANECODE. « Yanecode - Accueil. » [Consulté le 26 août 2026], Yanecode.
adresse : <https://yanecode.com/fr/>
- [2] YANECODE. « Robotic Yanecode. » [Consulté le 26 août 2026], Yanecode.
adresse : <https://robotic.yanecode.com/index.html>
- [3] YANECODE. « Academy Yanecode. » [Consulté le 26 août 2026], Yanecode.
adresse : <https://academy.yanecode.com/>
- [4] D. ANDERSON, *Kanban : Evolutionäres Change Management für IT-Organisationen*. dpunkt.verlag, 2024.
- [5] D. ANDERSON, *Kanban : Enclenchez le moteur d'amélioration continue de votre IT*, trad. par A. NICOLAS. Blue Hole Press, 2022.
- [6] ATLISSIAN. « Qu'est-ce que le Kanban ? » [Consulté le 26 août 2026], Atlassian. adresse : <https://www.atlassian.com/fr/agile/kanban>
- [7] G. A. SUPPORT. « Aviation Predictive Maintenance : Transforming Aircraft Safety and Efficiency. » [Consulté le 26 août 2026], Gen Aviation Support.
adresse : <https://genaviationsupport.com/aviation-predictive-maintenance-transforming-aircraft-safety-and-efficiency/>
- [8] R. RANPARA, « A Semantic and Ontology-Based Framework for Enhancing Interoperability and Automation in IoT Systems, » *Discover Internet of Things*, t. 5, n° 1, p. 1-12, 2025, [Consulté le 26 août 2026]. DOI : 10.1007/s43926-025-00122-8
- [9] E. REYNA. « TinyML : Low Power Edge Device for Predictive Maintenance. » [Consulté le 26 août 2026], ElectronicWings. adresse : <https://www.electronicwings.com/users/edgarreyna/projects/4208/tinyml-low-power-edge-device-for-predictive-maintenance>
- [10] R. SHRESTHA, « Anomaly Detection Based on LSTM and Autoencoders Using Federated Learning in Smart Electric Grid, » *Journal of Parallel and Distributed Computing*, t. 193, 2024, Article no. 104951. DOI : 10.1016/j.

jpd.c.2024.104951 adresse : <https://www.sciencedirect.com/science/article/pii/S0743731524001151>

- [11] R. N. ALGBURI, H. GAO et Z. AL-HUDA, « Detection of Weak Position Oscillations in the Industrial Robot by Singular Spectrum Analysis, » in *Developments of Artificial Intelligence Technologies in Computation and Robotics*, [Consulté le 26 août 2026], World Scientific, 2020, p. 1113-1122, ISBN : 9789811223334. DOI : 10.1142/9789811223334_0133
- [12] D. CHRISTIN, P. S. MOGRE et M. HOLLICK, « Survey on Wireless Sensor Network Technologies for Industrial Automation : The Security and Quality of Service Perspectives, » *Future Internet*, t. 2, n° 2, p. 96-125, 2010, [Consulté le 26 août 2026]. DOI : 10.3390/fi2020096 adresse : <https://www.mdpi.com/1999-5903/2/2/96>
- [13] G. PANDA, P. R. MAJHI et S. K. DHAL, « TCAE : A Temporal Contextual Autoencoder for Unsupervised Financial Fraud Detection, » *arXiv*, 2026, Preprint. adresse : <https://arxiv.org/abs/2512.19383>
- [14] C. LE. « IoT Edge Computing Fog Architecture. » [Consulté le 20 août 2026], Chanh Le Dev. adresse : <https://chanhle.dev/en/blog/iot-edge-computing-fog-architecture>
- [15] WOKWI. « Wokwi - ESP32, Arduino Simulator. » [Consulté le 10 août 2026], Visual Studio Marketplace. adresse : <https://marketplace.visualstudio.com/items?itemName=Wokwi.wokwi-vscode>
- [16] T. TEAM. « TDDA : Test-Driven Data Analysis. » [Consulté le 15 août 2026], TDDA Documentation. adresse : <https://tdda.readthedocs.io/en/latest/>
- [17] JSONIC. « Guide JSON pour IoT. » [Consulté le 12 août 2026], Jsonic. adresse : <https://jsonic.io/guides/json-iot>
- [18] SLACK. « Qu'est-ce que Slack ? » [Consulté le 26 août 2026], Slack. adresse : <https://slack.com/intl/fr-fr/help/articles/115004071768-Qu%E2%80%99est-ce-que-Slack>

- [19] M. PLUS. « ESP32 Dev Kit C V4 Documentation. » [Consulté le 22 août 2026], Manuals Plus. adresse : <https://manuals.plus/m/fce28436970f8f458a2bf-5600c672d20ef96d75c22688635cad333b465f91890>
- [20] AZ-DELIVERY. « ESP32 Dev Kit C V4 NodeMCU WLAN WiFi Development Board. » [Consulté le 26 août 2026], AZ-Delivery. adresse : <https://www.az-delivery.de/products/esp32-board-dev-kit-c-v4-nodemcu-wlan-wifi-development-board-unverlotet-mit-usb-c-anschluss-kompatibel-mit-arduino>

